Tutorial II

Desarrollo y compilación de una aplicación Android

React Native y TypeScript

Backend Python en Google Cloud, despliegue automático con
GitHub Actions y base de datos SQL

Taller de Innovación y Emprendimiento Tecnológico

Universidad Adolfo Ibáñez

Profesor:

Martín Céspedes Escuti

Ayudante Instructor:

Artemio Araya Day

17 de octubre de 2026

Índice

Definiciones 4

Google Cloud 4

¿Qué sistema operativo utiliza Google Cloud? 6

Servicios de Google Cloud 9

Firebase Authentication 12

Google Maps Platform 13

Google Cloud Run 15

Google Cloud Build 18

Google Artifact Registry 19

Google Cloud SQL 20

Google Secret Manager 21

Identity and Access Management (IAM) 22

Workload Identity Federation (WIF) 23

Google Auth Platform (OAuth) 24

GitHub Actions 25

Google Cloud Shell 26

Google Cloud Billing 27

Cloud Firestore 28

Preguntas y Respuestas 29

¿Qué es FastAPI? 30

¿Por qué GitHub Actions y no Google Cloud Build? 31

¿Que es PosgreSQL? 32

Aplicacion Móvil 35

Publicación de la aplicación móvil 37

Despliegue del backend 38

Funcionamiento de la aplicación móvil 45

Estructura del Repositorio de GitHub 49

Principales Herramientas de JavaScript utilizadas 51

Tutorial 53

Alcance y arquitectura 53
Nombres e identificadores de referencia 53
Descargar la Aplicación Web y subir el Proyecto a GitHub . . 54
Consultar los identificadores de GitHub 58
Crear el proyecto y activar la facturación 59
Habilitar las API de Google Cloud 60
Crear las cuentas de servicio 61
Vincular GitHub con la cuenta de servicio 62
Configurar las variables de autenticación en GitHub 63
Crear el repositorio en Artifact Registry 65
Configurar los permisos de Artifact Registry 67
Configurar las variables de Artifact Registry en GitHub 68
Crear una instancia de PostgreSQL en Cloud SQL 69
Crear la base de datos y el usuario en Cloud SQL 70
Crear la base de datos PostgreSQL 70
Crear el usuario de la aplicación 71
Conectar PostgreSQL mediante Cloud SQL Studio 72
Crear las tablas y configurar los permisos en PostgreSQL 74
Almacenar la contraseña de PostgreSQL en Secret Manager 77
Configurar los permisos de acceso a Secret Manager 79
Configurar las variables de conexión a PostgreSQL en GitHub . . . 81
Configurar Google Auth Platform y crear un cliente OAuth 83
Configurar la aplicación en Google Auth Platform 83
Crear un cliente OAuth para el backend 83
Configurar el ID de cliente OAuth en GitHub 85
Ejecutar el primer despliegue del backend 87
Configurar los permisos y el acceso público en Cloud Run 89
Permitir que la API invoque el motor privado 89
Habilitar el acceso público a la API 90
Configurar usuarios de prueba y OAuth para Android 92
Agregar usuarios de prueba en Google Auth Platform 92
Crear un cliente OAuth para Android 93

Preparar y compilar la aplicación Android 95
Extraer el proyecto e instalar las dependencias 95
Abrir el proyecto Android en Android Studio 96
Configurar JDK 17 para Gradle 96
Instalar los componentes del SDK de Android 97
Comprobar Java desde PowerShell 98
Corregir la selección de Java del daemon de Gradle 99
Compilar el APK de depuración 101
Ejecutar la versión de desarrollo 102

Referencias de las definiciones 103

Definiciones

Google Cloud

Google Cloud es un conjunto de servicios de computación en la nube que utiliza la misma infraestructura tecnológica empleada por Google en sus propios productos, como Google Search, Gmail y YouTube. Estos servicios permiten desarrollar, ejecutar y administrar aplicaciones mediante recursos de procesamiento, almacenamiento, bases de datos y redes disponibles a través de Internet (Google Cloud, s. f.).

Tiene sus orígenes en 2008, cuando Google presentó Google App Engine, un servicio que permitía desarrollar y ejecutar aplicaciones web utilizando su infraestructura tecnológica (McDonald, 2008). Posteriormente, en 2016, Google presentó oficialmente Google Cloud como una plataforma que integraba servicios de computación en la nube, herramientas de productividad y tecnologías empresariales (Greene, 2016).

Funciona mediante una infraestructura distribuida de centros de datos, servidores, redes de fibra óptica y sistemas de software que permiten ejecutar aplicaciones y almacenar información en distintas ubicaciones del mundo.

Precisamente combina servidores físicos, sistemas de almacenamiento, redes de comunicación y herramientas de administración automatizada. Su arquitectura utiliza sistemas desarrollados por Google, como Borg, encargado de administrar y distribuir las cargas de procesamiento entre servidores; Colossus, un sistema de almacenamiento distribuido; y Jupiter, una infraestructura de redes de alto rendimiento que permite la comunicación entre servidores dentro de los centros de datos. Estos componentes sustentan servicios como BigQuery y Spanner, permitiendo ejecutar aplicaciones, procesar información y almacenar datos sin que los usuarios deban administrar directamente la infraestructura física (Ayodele, 2022).

Respecto a los lenguajes de programación utilizados en su infraestructura, Google ha documentado el uso predominante de C++, Java, Python y Go en sus sistemas internos. La utilización de distintos lenguajes permite desarrollar componentes de software con diferentes características y requisitos técnicos. Sin embargo, Google no publica una descripción completa de los lenguajes empleados en cada componente de Google Cloud, por lo que no es posible determinar con precisión qué lenguaje utiliza cada uno de sus servicios internos. Esta información sobre los cuatro lenguajes principales procede de una publicación técnica oficial de Google de 2016 (Dinesh, 2016).

Geográficamente, Google Cloud dispone de una infraestructura distribuida internacionalmente mediante regiones y zonas de disponibilidad. Según la información publicada por Google en octubre de 2026, cuenta con 43 regiones y 130 zonas, distribuidas entre distintos continentes. Cada región constituye un área geográfica independiente que contiene varias zonas, diseñadas para proporcionar aislamiento frente a fallos y facilitar la implementación de aplicaciones redundantes. Estas ubicaciones se encuentran interconectadas mediante la infraestructura global de Google, que comprende aproximadamente 10 millones de kilómetros de fibra óptica terrestre y submarina, además de más de 200 ubicaciones de perímetro de red. Esta distribución permite seleccionar la ubicación de ejecución de las aplicaciones para reducir la latencia y mejorar su disponibilidad. En Chile, Google Cloud dispone de una región en Santiago, identificada como southamerica-west1 (Google Cloud, 2026).

¿Qué sistema operativo utiliza Google Cloud?

Google utiliza en sus servidores de producción un sistema operativo propio basado en Linux, diseñado específicamente para su infraestructura de computación en la nube. Según la documentación oficial de Google Cloud, este sistema operativo corresponde a una versión personalizada, reducida y reforzada de Linux, desarrollada para ejecutar los servicios de Google y reducir las vulnerabilidades de seguridad. A diferencia de las distribuciones convencionales de Linux, se encuentra adaptado a los requisitos de funcionamiento y seguridad de sus centros de datos (Kava, 2016) (Vergadia, 2022).

¿Qué diferencia hay entre los sistemas de almacenamiento de datos entre Google Cloud y Microsoft Azure?

Característica	Sistema de Bases de datos de Google Cloud (Cloud Firestore y Firebase Realtime Database)	Sistema de Bases de datos de Azure (Azure Cosmos DB, Azure SQL Database y Azure Database for PostgreSQL)	Consecuencia
Naturaleza del servicio	Plataforma integrada para aplicaciones web y móviles. Incluye bases de datos, autenticación, almacenamiento y servicios backend. (Firebase, s. f.-h)	Plataforma de computación en la nube que ofrece servicios independientes, como Azure Cosmos DB, Azure SQL Database y Azure Database for PostgreSQL. (Microsoft, s. f.-a)	Firebase reúne funcionalidades de desarrollo de aplicaciones en un mismo ecosistema. Azure permite seleccionar y combinar diferentes servicios según los requisitos de la aplicación. (Firebase, s. f.-h); (Microsoft, s. f.-a)
Modelo de almacenamiento	Cloud Firestore almacena documentos organizados en colecciones. Realtime Database utiliza una estructura de datos JSON. Ambas son bases de datos NoSQL. (Firebase, s. f.-b); (Firebase, s. f.-e)	Azure Cosmos DB proporciona almacenamiento NoSQL. Azure SQL Database y Azure Database for PostgreSQL ofrecen motores de bases de datos relacionales. (Microsoft, s. f.-a); (Microsoft, s. f.-e)	Las bases de datos NoSQL permiten almacenar documentos sin un esquema relacional rígido. Los motores relacionales facilitan las consultas SQL, las relaciones entre tablas y la integridad referencial. (Firebase, s. f.-b); (Microsoft, s. f.-d)
Conexión con aplicaciones	Dispone de SDK para aplicaciones móviles y web que permiten consultar y modificar datos directamente, bajo reglas de seguridad configuradas. (Firebase, s. f.-f)	Ofrece controladores, SDK y API para acceder a los motores de bases de datos. Las aplicaciones pueden utilizar servicios backend para administrar las operaciones. (Microsoft, s. f.-a)	Firebase facilita arquitecturas donde el cliente interactúa directamente con la base de datos. En Azure es habitual implementar un backend intermedio, especialmente para proteger credenciales y aplicar lógica de negocio. (Firebase, s. f.-f); (Microsoft, s. f.-a)
Sincronización y funcionamiento sin conexión	Firestore admite escuchas de cambios y persistencia local sin conexión. Realtime Database sincroniza modificaciones entre clientes conectados. (Firebase, s. f.-a); (Firebase, s. f.-e)	Azure Cosmos DB permite procesar modificaciones mediante Change Feed. La sincronización con interfaces móviles y el almacenamiento local dependen de la arquitectura implementada. (Microsoft, s. f.-b)	Firebase ofrece mecanismos integrados para actualizar información en aplicaciones móviles. Azure proporciona mecanismos de cambios que pueden combinarse con servicios adicionales para construir una sincronización equivalente. (Firebase, s. f.-a); (Microsoft, s. f.-b)
Consultas y procesamiento de datos	Firestore permite consultar documentos mediante filtros, índices y operaciones compatibles con su modelo de datos. (Firebase, s. f.-b)	Azure Cosmos DB admite consultas sobre documentos. Azure SQL Database y PostgreSQL permiten consultas relacionales mediante SQL. (Microsoft, s. f.-a); (Microsoft, s. f.-e)	Para datos documentales, Firestore y Cosmos DB pueden cumplir funciones similares. Para relaciones complejas entre tablas y consultas SQL, los motores relacionales de Azure ofrecen otro modelo de procesamiento. (Firebase, s. f.-b); (Microsoft, s. f.-e)
Autenticación y seguridad	Integra Firebase Authentication y reglas de seguridad de Firestore para controlar el acceso de los usuarios a los documentos. (Firebase, s. f.-f)	Permite combinar bases de datos con mecanismos de control de acceso y servicios de identidad, como Microsoft Entra External ID. (Microsoft, s. f.-c)	Firebase integra la identidad del usuario con las reglas de acceso a sus datos. En Azure pueden utilizarse servicios separados de autenticación, autorización y bases de datos. Ambos requieren configurar correctamente los permisos. (Firebase, s. f.-f); (Microsoft, s. f.-c)
Distribución geográfica	Firestore permite seleccionar ubicaciones regionales o multirregionales. Las configuraciones multirregionales replican información entre regiones determinadas. (Firebase, s. f.-c)	Azure Cosmos DB admite distribución global y replicación entre regiones. Azure SQL Database dispone de mecanismos de redundancia y replicación geográfica. (Microsoft, s. f.-a); (Microsoft, s. f.-e)	Ambas plataformas permiten configurar almacenamiento geográficamente distribuido. La latencia, disponibilidad, residencia de datos y costos dependen de las regiones y mecanismos de replicación seleccionados. (Firebase, s. f.-c); (Microsoft, s. f.-a)
Costos y administración	Firestore utiliza un modelo de cobro relacionado con lecturas, escrituras, eliminaciones, almacenamiento y transferencia de datos. Incluye cuotas gratuitas limitadas. (Firebase, s. f.-g)	Cosmos DB utiliza unidades de solicitud y modelos de capacidad configurables. Azure SQL y PostgreSQL ofrecen configuraciones de procesamiento y almacenamiento con costos asociados. (Microsoft, s. f.-a); (Microsoft, s. f.-e)	El costo depende de la cantidad y complejidad de las operaciones, los recursos contratados y el almacenamiento. Ninguna alternativa es universalmente más económica: la comparación requiere estimar la carga real de la aplicación. (Firebase, s. f.-g); (Microsoft, s. f.-a)

Fuente: elaboración propia a partir de la documentación oficial de Firebase, Google Cloud y Microsoft Learn. Comparación cualitativa entre los servicios de bases de datos y las funcionalidades relacionadas con su utilización en aplicaciones web y móviles. Las consecuencias se fundamentan en las fuentes citadas por fila. No constituye una medición experimental de rendimiento ni una comparación de precios para una carga de trabajo específica.

Característica	Ubuntu Server (Linux convencional)	Linux personalizado de Google	Consecuencia
Núcleo y sistema operativo	Utiliza el núcleo Linux integrado en una distribución de propósito general, con componentes y configuraciones mantenidos por Canonical. (Canonical, s. f.-b)	Utiliza un sistema operativo desarrollado internamente sobre Linux, personalizado para los servidores de producción de Google. (Kava, 2016)	Ambos están basados en Linux. La diferencia documentada reside en su especialización. Google no publica un inventario completo de las modificaciones internas de su núcleo. (Kava, 2016; Vergadia, 2022)
Componentes del sistema	Permite instalar, configurar y eliminar paquetes mediante herramientas de administración. Dispone de instalaciones mínimas y componentes opcionales. (Canonical, s. f.-b)	Google describe su sistema como una versión reducida de Linux, que elimina componentes innecesarios para sus servicios de producción. (Kava, 2016)	La reducción de componentes innecesarios puede disminuir la superficie de ataque y las tareas de mantenimiento. No se dispone de una lista pública completa de los componentes eliminados por Google. (Kava, 2016)
Ciberseguridad	Incluye mecanismos de protección como permisos de usuarios, actualizaciones de seguridad y herramientas de endurecimiento configurables. (Canonical, s. f.-a; Canonical, s. f.-b)	Google utiliza una versión reforzada de Linux e incorpora controles adicionales de aislamiento, autenticación y protección de las máquinas. (Google Cloud, 2024a; Vergadia, 2022)	Google integra la seguridad del sistema operativo con controles propios de su infraestructura. Esto no demuestra, por sí solo, una superioridad general frente a una instalación Linux correctamente reforzada. (Canonical, s. f.-a; Google Cloud, 2024a)
Arranque e integridad	Puede utilizar UEFI Secure Boot para verificar las firmas del cargador de arranque, el núcleo y sus módulos, según la configuración del equipo. (Canonical, s. f.-c)	En máquinas equipadas con Titan, utiliza arranque medido y credenciales vinculadas criptográficamente a versiones autorizadas del software. (Google Cloud, 2024b)	Google puede impedir que una máquina participe en su infraestructura si no inicia con el software autorizado. Titan constituye una protección adicional de hardware, no una modificación exclusiva del núcleo Linux. (Google Cloud, 2024b)
Acceso administrativo	Puede administrarse mediante terminal local o conexiones remotas SSH. Los permisos y mecanismos de autenticación dependen de la configuración del administrador. (Canonical, s. f.-b)	Restringe el acceso directo a las máquinas de producción. Utiliza interfaces limitadas, intermediarios seguros, autorizaciones y auditorías. (Google Cloud, 2026)	Google reduce la necesidad de acceso administrativo directo y puede exigir la aprobación de otro operador. La diferencia corresponde a políticas y herramientas de operación, no necesariamente al núcleo Linux. (Google Cloud, 2026)
Actualizaciones	Utiliza paquetes y repositorios de software. Puede aplicar automáticamente actualizaciones de seguridad mediante herramientas como unattended-upgrades. (Canonical, s. f.-a)	Google dispone de sistemas automatizados para actualizar el software, aplicar parches y verificar la integridad de los servidores de producción. (Google Cloud, 2024a)	Ambos permiten automatizar actualizaciones. Google integra este proceso con controles de identidad, verificación de integridad y recuperación de máquinas. (Canonical, s. f.-a; Google Cloud, 2024a)
Administración de cargas de trabajo	Puede ejecutar aplicaciones y utilizar herramientas de virtualización y orquestación, como Docker y Kubernetes, cuando se instalan y configuran. (Canonical, s. f.-b)	Google utiliza Borg para administrar la distribución y ejecución de servicios entre numerosos servidores de su infraestructura. (Google Cloud, 2024a)	Borg facilita la administración centralizada de cargas de trabajo a gran escala. Sin embargo, constituye una tecnología de orquestación adicional y no un sistema operativo diferente. (Google Cloud, 2024a)
Disponibilidad de información técnica	La distribución y sus componentes disponen de documentación pública, repositorios y mecanismos que permiten estudiar y modificar su configuración. (Canonical, s. f.-b)	Google publica información sobre el diseño y la seguridad de su infraestructura, pero no una especificación completa de su sistema operativo interno. (Google Cloud, 2024a; Kava, 2016)	No es posible establecer todas las diferencias de implementación, configuración del núcleo y componentes internos. Tampoco se puede realizar una comparación exhaustiva de rendimiento con la información pública disponible. (Google Cloud, 2024a; Kava, 2016)

Fuente: elaboración propia a partir de la documentación oficial de Canonical y Google Cloud. Las características y consecuencias se fundamentan en las fuentes citadas por fila. Se distingue entre las características del sistema operativo y los mecanismos adicionales de administración y seguridad de Google. Comparación técnica cualitativa, no una medición experimental de rendimiento.

Servicios de Google Cloud

Algunos servicios de Google Cloud para desarrollar aplicaciones móviles para Android con sistema de login con cuenta de google, base de datos y backend para procesamiento de información.

- **Google Cloud Run**: permite ejecutar los servicios automind-api y automind-inference mediante contenedores Docker. Ambos utilizan FastAPI y Uvicorn para atender las solicitudes, sin necesidad de administrar directamente servidores (Google Cloud, s. f.-d).

- **Google Cloud SQL**: proporciona el servicio administrado donde se aloja PostgreSQL. En el proyecto conserva las tablas de usuarios, modelos, permisos y solicitudes. Su herramienta Cloud SQL Studio permite ejecutar consultas y administrar los datos desde el navegador (Google Cloud, s. f.).

- **Google Secret Manager**: permite almacenar y administrar información sensible. En el proyecto conserva la contraseña de PostgreSQL, que se proporciona al servicio de la API mediante su configuración en Cloud Run (Google Cloud, s. f.).

- **Google Artifact Registry**: permite almacenar y administrar las imágenes Docker de la API y del motor de cálculo. GitHub Actions publica estas imágenes antes de desplegarlas en Google Cloud Run (Google Cloud, s. f.-a).

- **Identity and Access Management (IAM)**: permite administrar las cuentas de servicio, sus roles y sus permisos. En el proyecto controla el acceso a Cloud SQL, Secret Manager, Artifact Registry y Cloud Run, incluyendo la autorización de la API para invocar el motor privado (Google Cloud, s. f.-f).

- **Workload Identity Federation (WIF)**: es una funcionalidad de IAM que

permite a GitHub Actions obtener credenciales temporales para desplegar en Google Cloud mediante OIDC. Evita almacenar claves permanentes de cuentas de servicio en el repositorio (Google Cloud, s. f.).

- **Google Auth Platform (OAuth)**: permite configurar la información de la aplicación, la audiencia, los usuarios de prueba y los clientes OAuth necesarios para integrar el inicio de sesión con Google. El backend verifica directamente los tokens de identidad de Google mediante la biblioteca google-auth (Google for Developers, s. f.).

- **GitHub Actions**: es la plataforma de integración y despliegue continuos utilizada por el proyecto. Ejecuta las pruebas de Python, inicia PostgreSQL temporal para las pruebas de integración, construye las imágenes Docker, las publica en Artifact Registry y despliega los servicios en Cloud Run. Es un servicio de GitHub, externo a Google Cloud (GitHub, s. f.).

- **Google Cloud Shell**: proporciona una terminal accesible desde el navegador, con herramientas de línea de comandos de Google Cloud. Puede utilizarse como herramienta auxiliar para consultar recursos y ejecutar comandos de configuración. El despliegue automatizado del proyecto se realiza desde GitHub Actions (Google Cloud, s. f.-e).

- **Google Cloud Billing**: permite asociar una cuenta de facturación al proyecto, consultar el consumo de recursos y administrar los costos de los servicios de Google Cloud (Google Cloud, s. f.-b).

- **Firebase Authentication (servicio complementario)**: permite implementar autenticación mediante Google y otros proveedores. No se utiliza en la implementación actual: la aplicación inicia sesión con Google y el backend valida directamente sus tokens de identidad (Firebase, s. f.-b).

- **Cloud Firestore (servicio complementario)**: es una base de datos NoSQL que organiza la información en documentos y colecciones, con funciones

de sincronización en tiempo real y acceso sin conexión. No se utiliza para almacenar los datos de esta implementación, que emplea PostgreSQL en Cloud SQL (Firebase, s. f.-a).

- **Google Cloud Build (servicio complementario)**: permite ejecutar pruebas y construir imágenes Docker en Google Cloud, incluyendo procesos activados desde repositorios GitHub. No participa en el workflow actual: esas tareas se ejecutan mediante GitHub Actions (Google Cloud, s. f.-c).

- **Google Maps Platform (servicio complementario)**: proporciona API y SDK para incorporar mapas, buscar lugares, consultar información geográfica y calcular rutas y distancias. Puede incorporarse a funcionalidades geográficas de la aplicación; no forma parte del flujo de autenticación, almacenamiento y despliegue del backend descrito en este tutorial (Google Maps Platform, s. f.).

¿Qué son los tokens de identificación? Un token de autenticación de Firebase Authentication, denominado Firebase ID Token, es una credencial digital temporal que permite identificar de manera segura a un usuario autenticado en una aplicación. Consiste en un JSON Web Token (JWT) firmado digitalmente, que contiene información como el identificador único del usuario (UID), el proyecto de Firebase al que pertenece y su fecha de expiración.

Este token puede enviarse mediante HTTPS a un servidor backend, donde se verifica su autenticidad, integridad y vigencia antes de autorizar el acceso a los recursos de la aplicación (Firebase, s. f.-b).

Los tokens de ID tienen una duración aproximada de una hora y pueden renovarse mediante un token de actualización, permitiendo mantener la sesión del usuario sin solicitar nuevamente sus credenciales en cada operación (Firebase, s. f.-a).

Google Maps Platform

Google Maps Platform es una plataforma de servicios de Google que proporciona interfaces de programación de aplicaciones (API) y kits de desarrollo de software (SDK) para incorporar mapas interactivos, información geográfica, búsqueda de lugares y cálculo de rutas en aplicaciones web y móviles. Estos servicios pueden habilitarse y administrarse mediante proyectos de Google Cloud (Google Maps Platform, s. f.).

- **Maps JavaScript API**: permite incorporar mapas interactivos y personalizables en páginas web mediante JavaScript.
- **Maps Embed API**: permite insertar mapas de Google en páginas web mediante elementos HTML.
- **Maps Static API**: permite generar imágenes estáticas de mapas a partir de solicitudes HTTP.
- **Places API**: permite buscar lugares y obtener información sobre establecimientos, direcciones y puntos de interés.
- **Routes API**: permite calcular rutas, distancias y tiempos de desplazamiento considerando diferentes medios de transporte y condiciones de tráfico.
- **Route Optimization API**: permite optimizar recorridos que incluyen múltiples destinos y vehículos.
- **Geocoding API**: permite convertir direcciones postales en coordenadas geográficas y viceversa.
- **Geolocation API**: permite estimar la ubicación geográfica de un dispositivo utilizando información de redes Wi-Fi y antenas de telefonía móvil.
- **Street View Static API**: permite obtener imágenes panorámicas de calles y ubicaciones mediante solicitudes HTTP.
- **Map Tiles API**: proporciona mosaicos cartográficos bidimensionales, tridimensionales y de Street View para construir visualizaciones geográficas.
- **Elevation API**: permite consultar la elevación de puntos geográficos respecto al nivel del mar.
- **Roads API**: permite asociar coordenadas GPS con las carreteras de la red vial y consultar información sobre sus segmentos.
- **Time Zone API**: permite determinar la zona horaria correspondiente a una ubicación geográfica.

- **Address Validation API**: permite validar direcciones postales e identificar errores o información incompleta.
- **Air Quality API**: permite consultar datos sobre la calidad del aire y los contaminantes atmosféricos de determinadas ubicaciones.
- **Weather API**: permite consultar condiciones meteorológicas actuales, pronósticos e información meteorológica histórica.

Fuente: documentación oficial de Google Maps Platform (Google Maps Platform, s. f.).

Google Cloud Run

Es un servicio de computación sin servidor (*serverless*) que permite ejecutar aplicaciones, API, funciones y procesos informáticos mediante contenedores aislados, sin administrar directamente la infraestructura física.

Proporciona recursos configurables de CPU y memoria, conexiones HTTPS y mecanismos de escalado automático que ajustan la cantidad de instancias según la demanda. Permite ejecutar código desarrollado en distintos lenguajes de programación y admite servicios HTTP, trabajos y procesos en segundo plano (Google Cloud, s. f.).

¿Qué es un contenedor aislado? Un contenedor aislado es un entorno que permite ejecutar una aplicación junto con sus bibliotecas y dependencias, separándola de otras aplicaciones que utilizan el mismo servidor. Los contenedores pueden compartir el núcleo del sistema operativo, pero mantienen separados sus procesos y archivos, evitando conflictos entre las aplicaciones y permitiendo aprovechar los recursos de una misma infraestructura (Google Cloud, s. f.).

¿Qué son las instancias? Una instancia de contenedor en Google Cloud Run es un entorno de ejecución aislado que ejecuta una copia de una aplicación mediante recursos computacionales configurables, como CPU y memoria RAM. Cada instancia puede procesar una o varias solicitudes simultáneamente, dependiendo de su configuración.

Google Cloud Run permite aumentar o disminuir automáticamente la cantidad de instancias según la demanda del servicio, pudiendo reducirlas a cero cuando no existen solicitudes y la configuración lo permite (Google Cloud, s. f.).

La configuración de un número máximo de instancias en Google Cloud Run permite limitar la cantidad de entornos de ejecución que pueden funcionar simultáneamente para atender las solicitudes de una aplicación. Este límite contribuye a controlar el consumo de recursos computacionales, reducir el riesgo de costos inesperados y evitar la sobrecarga de servicios externos, como las bases de datos.

Cuando se alcanza el máximo configurado, Cloud Run intenta atender las solicitudes adicionales utilizando la capacidad disponible, aunque algunas pueden ser rechazadas si esta resulta insuficiente. El límite puede superarse temporalmente en determinadas condiciones operativas, por lo que no constituye una restricción absoluta del consumo ni del costo del servicio (Google Cloud, s. f.).

Google Cloud Build

Es un servicio administrado de Google Cloud que permite ejecutar procesos de compilación y construcción de aplicaciones utilizando la infraestructura de Google. Puede obtener el código fuente desde repositorios como GitHub, GitLab y Bitbucket, así como desde Google Cloud Storage. Su función principal consiste en transformar el código fuente y sus archivos de configuración en artefactos de software que puedan utilizarse posteriormente para ejecutar o distribuir una aplicación, como imágenes de contenedores Docker.

El proceso de construcción se define mediante un archivo de configuración en formato YAML o JSON, donde se especifican las instrucciones que debe ejecutar Cloud Build. Estas instrucciones se organizan en pasos independientes, cada uno ejecutado dentro de un contenedor Docker. Los pasos pueden incluir la instalación de dependencias, ejecución de pruebas, análisis del código y construcción de imágenes Docker. Una vez generados, los artefactos pueden almacenarse en Google Artifact Registry para su posterior utilización en otros servicios de Google Cloud.

Google Cloud Build permite ejecutar estos procesos manualmente o automatizarlos mediante activadores vinculados a eventos de repositorios, como la incorporación de cambios en GitHub. De esta manera, puede formar parte de un sistema de integración continua y entrega continua (CI/CD). Por ejemplo, cuando se modifica el código de un backend desarrollado en Python y FastAPI, Cloud Build puede construir una nueva imagen Docker y almacenarla en Artifact Registry. Posteriormente, esta imagen puede desplegarse en Google Cloud Run mediante un proceso de despliegue configurado para ello. Por tanto, Cloud Build se encarga principalmente de construir y preparar el software, mientras que Cloud Run permite ejecutarlo y alojarlo como servicio (Google Cloud, s. f.).

Google Artifact Registry

Google Artifact Registry es un servicio administrado de Google Cloud que permite almacenar, organizar y administrar artefactos de software generados durante el desarrollo y la construcción de aplicaciones. Un artefacto es un archivo producido o utilizado durante estos procesos, como una imagen de contenedor Docker o un paquete de software. Estos elementos se almacenan en repositorios que pueden configurarse para distintos formatos, como Docker, Python, Java y Node.js. Cada repositorio posee una ubicación regional o multirregional y permite centralizar los archivos necesarios para distribuir o desplegar aplicaciones.

El servicio permite administrar múltiples repositorios dentro de un mismo proyecto de Google Cloud y controlar quién puede acceder a ellos mediante Identity and Access Management (IAM). Los permisos determinan qué usuarios o cuentas de servicio pueden consultar, descargar, publicar o eliminar artefactos. Además, Artifact Registry permite almacenar diferentes versiones de las imágenes Docker, identificarlas mediante etiquetas y administrar su conservación mediante políticas de limpieza. También puede integrarse con Artifact Analysis para examinar las imágenes almacenadas y detectar vulnerabilidades conocidas en sus componentes de software.

Google Artifact Registry se integra con servicios como Google Cloud Build y Google Cloud Run para facilitar la integración y entrega continuas (CI/CD). Por ejemplo, cuando se modifica el código fuente de un backend desarrollado en Python y FastAPI, Cloud Build puede construir una imagen Docker que contenga la aplicación, sus dependencias y las instrucciones necesarias para ejecutarla. Posteriormente, esta imagen puede publicarse en un repositorio de Artifact Registry, desde donde Cloud Run puede recuperarla para desplegar y ejecutar el backend. De esta manera, Cloud Build se encarga de construir la imagen, Artifact Registry de almacenarla y administrarla, y Cloud Run de ejecutarla como un servicio accesible mediante solicitudes HTTP (Google Cloud, s. f.).

Google Cloud SQL

Google Cloud SQL es un servicio administrado de bases de datos relacionales que permite ejecutar motores como PostgreSQL sin administrar directamente el servidor donde se alojan. Proporciona herramientas para configurar el almacenamiento, las conexiones, los usuarios y las copias de seguridad de las instancias (Google Cloud, s. f.).

En este proyecto aloja la base PostgreSQL que conserva los usuarios, modelos, permisos y solicitudes. La API consulta y modifica esa información mediante SQLAlchemy y el conector de Cloud SQL. Cloud SQL Studio permite ejecutar instrucciones SQL desde el navegador para preparar las tablas, consultar registros y administrar los accesos. La base de datos permanece independiente de las revisiones desplegadas en Cloud Run.

Google Secret Manager

Google Secret Manager es un servicio que permite almacenar y administrar información sensible, como contraseñas, claves de API y certificados. Organiza los secretos mediante versiones y permite controlar su acceso mediante IAM (Google Cloud, s. f.).

En este proyecto almacena la contraseña del usuario de PostgreSQL. La configuración de Cloud Run proporciona una versión de ese secreto al servicio automind-api, cuya cuenta de ejecución debe tener autorización para acceder a ella. Así, la contraseña no necesita incorporarse al código fuente ni a la imagen Docker.

Identity and Access Management (IAM)

Identity and Access Management, denominado IAM, permite administrar quién puede acceder a los recursos de Google Cloud y qué operaciones puede realizar. Los permisos se agrupan en roles que se conceden a identidades, como usuarios y cuentas de servicio, sobre los recursos correspondientes (Google Cloud, s. f.).

En este proyecto se utilizan cuentas de servicio diferentes para desplegar el backend, ejecutar la API y ejecutar el motor de cálculo. Sus roles permiten publicar imágenes, actualizar servicios y acceder a Cloud SQL o Secret Manager. Además, el rol de invocación de Cloud Run autoriza a la API a comunicarse con el motor privado.

Workload Identity Federation (WIF)

Workload Identity Federation es una funcionalidad de IAM que permite autorizar cargas de trabajo externas mediante identidades federadas. Puede utilizar tokens emitidos por proveedores compatibles, como GitHub Actions mediante OIDC, para obtener acceso temporal a recursos de Google Cloud sin utilizar claves permanentes de cuentas de servicio (Google Cloud, s. f.).

En este proyecto se configura un grupo de identidades y un proveedor para GitHub. Las condiciones del proveedor restringen las identidades aceptadas según atributos del repositorio y de la ejecución. Posteriormente, el workflow autorizado obtiene credenciales temporales de la cuenta de despliegue para publicar las imágenes y actualizar Cloud Run.

Google Auth Platform (OAuth)

Google Auth Platform permite configurar la aplicación que solicita acceso mediante OAuth, incluyendo su información, la audiencia, los usuarios de prueba y los clientes OAuth. Esta configuración permite identificar la aplicación durante el inicio de sesión con Google (Google for Developers, s. f.).

En este proyecto se configura un cliente OAuth de Android asociado al nombre del paquete y a la huella SHA-1 del certificado de firma. También se utiliza un cliente OAuth de tipo aplicación web, cuyo identificador se configura como audiencia esperada por el backend. La API verifica los tokens de identidad de Google mediante google-auth, sin utilizar Firebase Authentication.

GitHub Actions

GitHub Actions es una plataforma de integración continua y entrega continua que permite automatizar pruebas, construcciones y despliegues mediante workflows definidos en archivos YAML dentro del repositorio. Puede ejecutar estos procesos ante eventos como actualizaciones de ramas, pull requests o activaciones manuales. Es un servicio de GitHub, externo a Google Cloud (GitHub, s. f.).

En este proyecto ejecuta las pruebas del backend y prepara una base temporal de PostgreSQL para las pruebas de integración, utilizando los archivos SQL. Después de superar las verificaciones, construye las imágenes Docker, se autentica en Google Cloud mediante WIF, publica las imágenes en Artifact Registry y despliega los servicios en Cloud Run.

Google Cloud Shell

Google Cloud Shell proporciona un entorno de terminal accesible desde el navegador. Incluye herramientas de desarrollo y administración, como la interfaz de línea de comandos de Google Cloud, para consultar recursos y ejecutar operaciones sobre los proyectos según los permisos del usuario (Google Cloud, s. f.).

Puede utilizarse como herramienta auxiliar para realizar configuraciones iniciales, consultar identificadores y ejecutar comandos administrativos del tutorial. El despliegue automatizado implementado se ejecuta en GitHub Actions, por lo que no requiere mantener una sesión de Cloud Shell abierta.

Google Cloud Billing

Google Cloud Billing permite administrar la facturación de los recursos utilizados en Google Cloud. Una cuenta de facturación puede vincularse a los proyectos correspondientes para habilitar el consumo de servicios que requieren facturación y consultar los costos asociados (Google Cloud, s. f.).

En el tutorial permite revisar los cargos generados por los servicios del proyecto, como Cloud SQL, Cloud Run y Artifact Registry. También permite configurar presupuestos y alertas para supervisar el gasto. Estas alertas informan sobre el consumo; no constituyen por sí mismas un límite automático que detenga los servicios.

Cloud Firestore

Cloud Firestore es un servicio de base de datos NoSQL que organiza la información en documentos y colecciones. Proporciona herramientas para consultar y modificar datos, recibir actualizaciones en tiempo real y trabajar sin conexión mediante sus SDK compatibles (Firebase, s. f.).

Se presenta como un servicio complementario, pero no se utiliza en la implementación actual del proyecto. Los datos de usuarios, modelos, permisos y solicitudes se almacenan en PostgreSQL mediante Cloud SQL. Por tanto, las colecciones y las reglas de seguridad de Firestore no forman parte del almacenamiento implementado.

Preguntas y Respuestas

¿Qué es FastAPI?

FastAPI es un framework de Python para desarrollar APIs, es decir, interfaces mediante las cuales una aplicación móvil puede solicitar información o ejecutar operaciones en un servidor. Permite definir rutas HTTP y describir los datos de entrada y salida mediante anotaciones de tipos de Python. A partir de estas definiciones, facilita la validación de datos con Pydantic y genera documentación interactiva basada en OpenAPI. Esto permite establecer un contrato de comunicación entre la aplicación móvil y el backend, con respuestas habitualmente en formato JSON (FastAPI, s. f.-b).

En un backend para aplicaciones móviles, FastAPI permite organizar las operaciones del servidor, como registrar información, consultar bases de datos, procesar solicitudes e integrar servicios externos. También proporciona herramientas para implementar autenticación y autorización, por ejemplo, mediante tokens y esquemas OAuth2. El desarrollador debe programar la verificación de identidad y las reglas de acceso correspondientes; utilizar el framework no aplica estas restricciones automáticamente. Las conexiones con bases de datos se implementan mediante bibliotecas o controladores compatibles (FastAPI, s. f.-c; FastAPI, s. f.-d).

Para utilizar este backend desde dispositivos móviles, se despliega en infraestructura accesible por red, como un servicio de contenedores o una máquina virtual en la nube. FastAPI define el comportamiento de la API, mientras que un servidor ASGI, como Uvicorn, ejecuta la aplicación y atiende las peticiones. En producción también deben configurarse HTTPS, el inicio y reinicio de procesos, los recursos disponibles y, cuando sea necesario, varias instancias para atender la demanda. Por tanto, FastAPI forma parte del backend, mientras que la plataforma de despliegue proporciona la infraestructura necesaria para mantenerlo disponible (FastAPI, s. f.-a).

¿Por qué GitHub Actions y no Google Cloud Build?

Se utilizó GitHub Actions para mantener la automatización junto al código fuente del proyecto. El archivo .github/workflows/deploy.yml define las pruebas de la API, las calculadoras y la integración con PostgreSQL, así como la construcción de las imágenes Docker y su despliegue en Cloud Run. Esto permite revisar los cambios, consultar los resultados de las pruebas y controlar el despliegue desde el mismo repositorio.

GitHub Actions se autentica en Google Cloud mediante OIDC y Workload Identity Federation, utilizando credenciales temporales. Después publica las imágenes en Artifact Registry y actualiza los servicios de Cloud Run. Por tanto, la arquitectura implementada cubre el proceso de integración y despliegue sin requerir Cloud Build como servicio adicional.

Google Cloud Build también permite ejecutar pruebas, construir imágenes y automatizar despliegues a partir de repositorios GitHub. Su utilización habría requerido configurar la conexión con el repositorio, los disparadores, los permisos y las instrucciones de construcción. La elección de GitHub Actions responde a la centralización del flujo de trabajo en GitHub y a la configuración implementada, sin implicar que Cloud Build sea incompatible o técnicamente inferior.

¿Que es PosgreSQL?

PostgreSQL es un sistema de gestión de bases de datos relacionales que permite almacenar, organizar y consultar información mediante el lenguaje SQL. En la arquitectura del backend, PostgreSQL se utiliza para mantener la información persistente de usuarios, modelos, permisos y solicitudes. Su ejecución en producción se realiza mediante Google Cloud SQL, mientras que la API desarrollada con FastAPI establece la comunicación con la base de datos para realizar consultas y operaciones autorizadas.

Tecnología	¿Qué es?	Aplicaciones generales	Responsabilidad técnica	Ubicación en la arquitectura	Persistencia de datos	Relación con el despliegue del backend
SQL	Es un lenguaje declarativo utilizado para definir, consultar, manipular y controlar datos en sistemas de bases de datos relacionales. (PostgreSQL Global Development Group, s. f.-b)	Se utiliza para crear tablas, consultar información, insertar, actualizar y eliminar registros, establecer restricciones y administrar permisos. (PostgreSQL Global Development Group, s. f.-b)	Define las operaciones que debe realizar un gestor de bases de datos mediante instrucciones como CREATE, SELECT, INSERT, UPDATE y DELETE. (PostgreSQL Global Development Group, s. f.-b)	Se emplea en aplicaciones, herramientas de administración, scripts y consultas enviadas a los motores de bases de datos relacionales. (PostgreSQL Global Development Group, s. f.-b)	No constituye un sistema de almacenamiento. Sus instrucciones son interpretadas y ejecutadas por un gestor que administra los datos. (PostgreSQL Global Development Group, s. f.-b)	Permite definir esquemas, ejecutar migraciones y realizar consultas desde el backend. Las instrucciones pueden ejecutarse durante el despliegue o durante la operación. (PostgreSQL Global Development Group, s. f.-b)
PostgreSQL	Es un sistema gestor de bases de datos objeto-relacional, de código abierto, que utiliza y amplía el lenguaje SQL. (PostgreSQL Global Development Group, s. f.-a)	Se utiliza en aplicaciones web, sistemas empresariales, plataformas transaccionales, análisis de datos y aplicaciones geoespaciales. (PostgreSQL Global Development Group, s. f.-a)	Ejecuta consultas SQL, administra transacciones, controla accesos, aplica restricciones y mantiene la integridad y consistencia de la información. (PostgreSQL Global Development Group, s. f.-a)	Puede instalarse en servidores locales, máquinas virtuales, contenedores o infraestructura en la nube, y actúa como motor de almacenamiento. (PostgreSQL Global Development Group, s. f.-a)	Almacena físicamente datos, tablas e índices. Utiliza mecanismos transaccionales y de recuperación para preservar la integridad. (PostgreSQL Global Development Group, s. f.-a)	Funciona como servidor de base de datos al que se conecta el backend. Su despliegue puede gestionarse por separado, con almacenamiento persistente. (PostgreSQL Global Development Group, s. f.-a)
Google Cloud SQL	Es un servicio administrado de Google Cloud que permite alojar bases de datos relacionales utilizando PostgreSQL, MySQL o SQL Server. (Google Cloud, s. f.)	Se utiliza para alojar bases de datos de aplicaciones web, móviles y empresariales que requieren almacenamiento relacional administrado en la nube. (Google Cloud, s. f.)	Gestiona infraestructura, mantenimiento, actualizaciones, respaldos y opciones de alta disponibilidad, según la configuración del servicio. (Google Cloud, s. f.)	Opera en la infraestructura de Google Cloud y puede conectarse con aplicaciones ejecutadas en Cloud Run, Compute Engine y otros entornos compatibles. (Google Cloud, s. f.)	Conserva los datos mediante almacenamiento persistente administrado y ofrece funciones de respaldo y recuperación configurables. (Google Cloud, s. f.)	Permite desplegar y actualizar el backend independientemente de la instancia de base de datos, manteniendo la conexión mediante credenciales y configuración de red. (Google Cloud, s. f.)

Nota: SQL es el lenguaje utilizado para interactuar con bases de datos relacionales; PostgreSQL es un sistema gestor que interpreta y ejecuta instrucciones SQL; Google Cloud SQL es un servicio administrado que permite alojar motores relacionales, incluido PostgreSQL, en la infraestructura de Google Cloud. (PostgreSQL Global Development Group, s. f.-b) (PostgreSQL Global Development Group, s. f.-a) (Google Cloud, s. f.)

Aplicacion Móvil

La aplicación móvil, identificada en la interfaz como **AutoMind**, actúa como cliente del proyecto **MLStore**. Permite consultar una Store de 24 modelos organizados en Mercado Inmobiliario, Telecomunicaciones, Minería y Forestal. En cada área se presentan seis denominaciones: Árbol de Decisión KS, Árbol de Decisión CART, Red Neuronal, Regresión Lineal Múltiple, Holt Winters y Prophet de Meta. Cada ficha tiene una descripción y un ID numérico único de ocho dígitos, del 10000001 al 10000024. En esta implementación, todos los modelos son calculadoras que suman tres entradas; sus nombres no implican que se hayan entrenado o ejecutado esos algoritmos de aprendizaje codigo-app,codigo-backend.

El usuario debe iniciar sesión con una cuenta de Google Gmail para consultar el catálogo, revisar sus modelos habilitados y enviar solicitudes de acceso. La sección *Mis modelos* muestra los IDs autorizados para la identidad autenticada, mientras que la Store permite consultar las fichas y solicitar otros accesos. La solicitud queda pendiente hasta que el administrador concede el permiso en la base de datos. Para utilizar una calculadora habilitada, el usuario introduce tres valores y solicita el cálculo al backend codigo-app,codigo-backend.

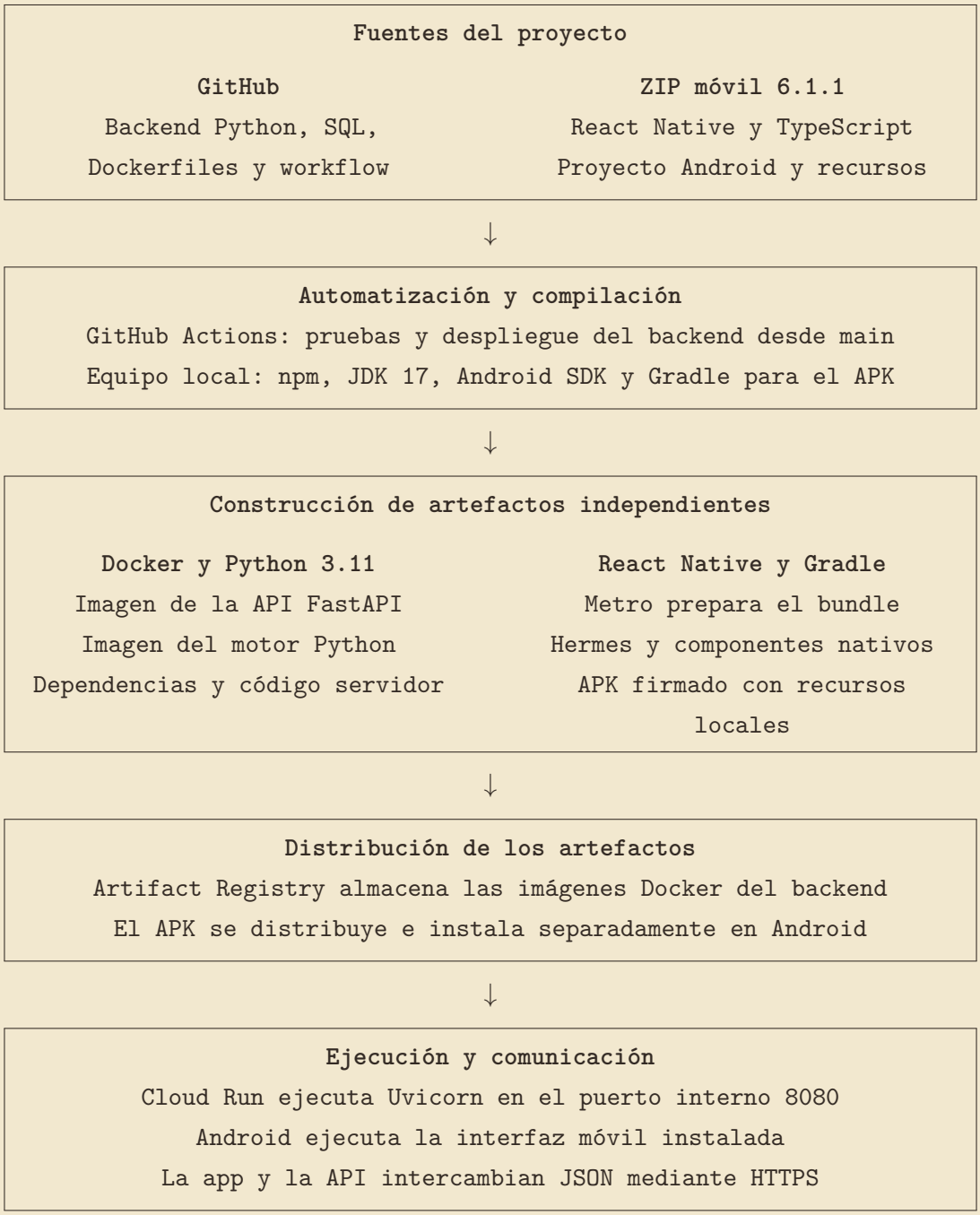
La interfaz de la versión ligera 6.1.1 utiliza React Native y TypeScript, con componentes nativos Text y View, fondo beige, líneas de papel y la fuente local Latin Modern Mono. Conserva el encabezado con efecto de máquina de escribir y muestra la fórmula $y = x_1 + x_2 + x_3$. Skia y RaTeX se sustituyeron por texto y elementos nativos sencillos. La aplicación se ejecuta como una app instalada en Android, sin WebView ni una página HTML integrada codigo-app,react-native.

El backend utiliza FastAPI en Python y almacena los datos en PostgreSQL. La base storeml_database se aloja en storeml-pg, una instancia de Cloud SQL, y contiene cuatro tablas de aplicación. Una API atiende al cliente móvil y un segundo servicio privado ejecuta las calculadoras Python. GitHub Actions construye dos imágenes Docker, las publica en Artifact Registry y las despliega en Cloud Run. Por separado, el frontend se

compila como un APK de Android. El ZIP

StoreML-App-Ligera-6.1.1.zip contiene la carpeta mobile y su documentación; utiliza la API existente y conserva su configuración de Google, paquete y certificado de depuración codigo-app,codigo-backend,fastapi-docker.

Publicación de la aplicación móvil



codigo-app,codigo-backend,docker,github-actions,hermes,artifact-registry

Despliegue del backend

Repositorio GitHub
Código Python 3.11, archivos SQL y Dockerfiles
Workflow de pruebas y despliegue
Rama principal main

↓

Integración continua: GitHub Actions
Pruebas de la API, las calculadoras y la configuración
PostgreSQL 17 temporal para pruebas de integración
Ejecución de los tres archivos SQL en esa base temporal

↓

Construcción de las imágenes Docker
Se ejecuta únicamente después de superar las pruebas
Dos imágenes independientes: api e inference
Python 3.11, código y dependencias de cada servicio

↓

Autenticación y publicación en Artifact Registry
Credenciales temporales mediante OIDC y WIF

Imagen api	**Imagen inference**
FastAPI y Uvicorn	FastAPI y Uvicorn
SQLAlchemy y conector Cloud SQL	Motor y calculadoras Python

↓

Despliegue en Google Cloud Run

automind-api	**automind-inference**
API accesible por HTTPS	Motor protegido por IAM
Valida identidad y permisos	Ejecuta calculadoras Python
Uvicorn en puerto interno 8080	Uvicorn en puerto interno 8080

↓

Funcionamiento en producción
Peticiones HTTPS y respuestas JSON, sin WebSockets
Cloud SQL conserva la base PostgreSQL de producción
Secret Manager proporciona la contraseña a la API
El despliegue no ejecuta los SQL sobre Cloud SQL

codigo-backend,docker,github-actions,artifact-registry

Etapa 1. GitHub: almacenamiento del código fuente.

El repositorio público Tutorial-App-II almacena el código fuente del backend, desarrollado en Python 3.11, junto con los Dockerfiles, los archivos SQL y la configuración del despliegue. El backend se organiza en dos componentes: api/, que contiene la API desarrollada con FastAPI, e inference/, que contiene el servicio de cálculo y las calculadoras Python organizadas por área.

La carpeta sql/ contiene tres scripts: 01-esquema.sql, que define las tablas y sus restricciones; 02-catalogo-y-accesos.sql, que carga el catálogo y los accesos iniciales; y 03-privilegios-app.sql, que configura los privilegios del usuario de la aplicación.

Estos scripts se utilizan para preparar la base de datos y también se ejecutan automáticamente sobre una base temporal durante las pruebas de GitHub Actions. Sin embargo, el workflow no los ejecuta sobre la base de producción en Cloud SQL. Por tanto, actualizar el código del backend y modificar el esquema de producción son operaciones independientes. La rama main proporciona el código utilizado para desplegar los servicios codigo-backend.

Etapa 2. GitHub Actions: integración continua y pruebas.

El workflow .github/workflows/deploy.yml automatiza las pruebas y el despliegue del backend. En su ejecución normal, las actualizaciones de main, los pull requests dirigidos a esa rama y las ejecuciones manuales activan las verificaciones configuradas.

Las pruebas se ejecutan con Python 3.11 e incluyen la API, las 24 calculadoras Python, la validación de las variables de despliegue y la integración con PostgreSQL. Para estas últimas, GitHub Actions inicia una instancia temporal de PostgreSQL 17 con una base de pruebas denominada automind_test.

Las pruebas de integración leen los tres archivos SQL del repositorio: los dos primeros preparan las tablas, el catálogo y los accesos iniciales; el tercero se utiliza para comprobar los privilegios del usuario de la aplicación. Estas operaciones se realizan sobre la base temporal de pruebas y no modifican Cloud SQL.

Por ello, los archivos SQL son necesarios para completar el workflow tal como está implementado. Aunque la construcción de las imágenes Docker no los necesita directamente, su ausencia provoca fallos en las pruebas e impide alcanzar la etapa de despliegue.

Los pull requests ejecutan las pruebas sin desplegar. El despliegue se permite desde main mediante un push o una ejecución manual, siempre que las pruebas terminen correctamente. Además, el workflow comprueba que el commit siga siendo el actual de main antes de continuar con su despliegue codigo-backend,github-actions.

Etapa 3. Docker: construcción de las imágenes del backend.

Después de superar las pruebas, GitHub Actions utiliza Docker para construir dos imágenes independientes a partir de los Dockerfiles del repositorio. Ambas emplean python:3.11-slim como imagen base.

La imagen api incorpora FastAPI, Uvicorn, SQLAlchemy, el controlador de PostgreSQL, el conector de Cloud SQL, las dependencias de autenticación y el código de la API. Su Dockerfile no copia la carpeta sql/ dentro de la imagen.

La imagen inference incorpora el servicio FastAPI del motor, Uvicorn, sus dependencias y los archivos de las calculadoras Python. Durante su construcción también ejecuta las pruebas de las calculadoras.

FastAPI define las rutas, valida los datos y organiza las operaciones de cada servicio. Uvicorn es el servidor ASGI que ejecuta estas aplicaciones y atiende las solicitudes dentro de los contenedores. La configuración actual utiliza peticiones HTTP y respuestas JSON; no implementa rutas WebSocket.

Esta separación permite ejecutar la API y el motor de cálculo en servicios independientes de Cloud Run codigo-backend,docker.

Etapa 4. Artifact Registry: publicación de las imágenes Docker.

Una vez construidas las imágenes, GitHub Actions se autentica en Google Cloud mediante OpenID Connect (OIDC) y Workload Identity Federation (WIF). Este mecanismo permite obtener credenciales temporales asociadas a la cuenta de servicio de despliegue, sin almacenar una clave permanente de esa cuenta en el repositorio.

Posteriormente, el workflow publica las imágenes api e inference en el repositorio de Artifact Registry configurado para el proyecto, denominado storeml por defecto.

Cada imagen se etiqueta con el SHA del commit de GitHub. Después de publicarlas, el workflow obtiene sus referencias mediante digest y las utiliza para desplegar en Cloud Run. Esto permite identificar el contenido exacto de las imágenes utilizadas por cada despliegue codigo-backend,artifact-registry,wif.

Etapa 5. Cloud Run: despliegue y ejecución del backend.

GitHub Actions despliega las imágenes en dos servicios independientes de Google Cloud Run. Primero actualiza automind-inference y obtiene su URL. Después configura y despliega automind-api, incluyendo la dirección del motor entre sus variables de entorno.

Ambos servicios ejecutan sus aplicaciones FastAPI mediante Uvicorn en el puerto interno 8080. La aplicación móvil se comunica con la API mediante HTTPS, utilizando peticiones GET y POST y respuestas JSON. La implementación actual no mantiene conexiones WebSocket.

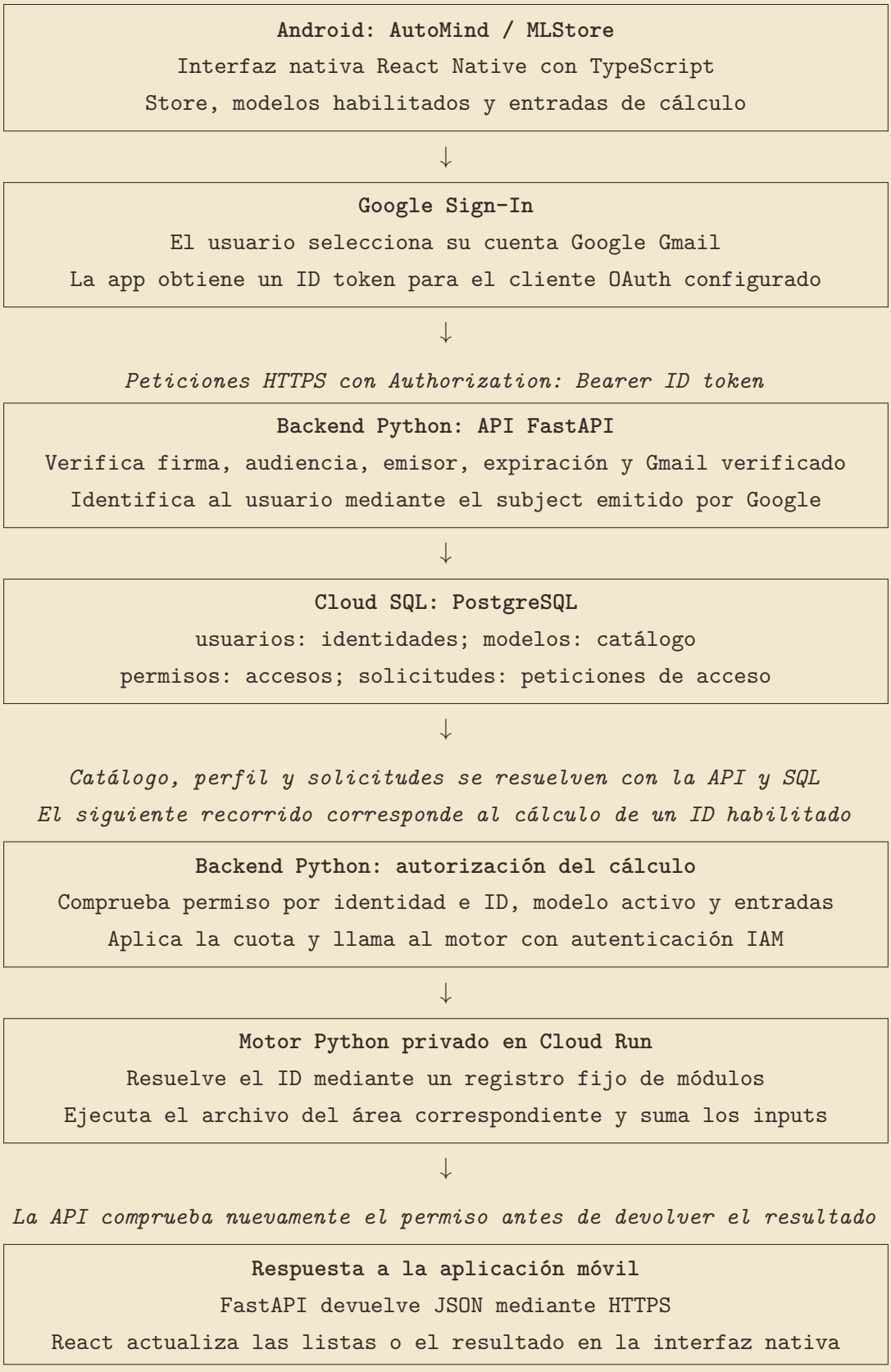
El servicio automind-api es accesible desde Internet, pero sus operaciones exigen un token de identidad válido de Google. La API verifica la identidad del usuario y consulta sus permisos antes de autorizar los cálculos. Para acceder a PostgreSQL, utiliza el conector de Cloud SQL y su cuenta de servicio de ejecución. La contraseña de la base de datos se incorpora mediante Secret Manager.

El servicio automind-inference permanece protegido mediante Cloud Run IAM. La API lo invoca mediante una solicitud HTTPS autenticada con un token de identidad destinado a ese servicio. La cuenta de ejecución de la API debe tener el permiso de invocación sobre el motor privado.

Los despliegues generan nuevas revisiones de los servicios de Cloud Run. El workflow no recrea la base de datos de producción ni ejecuta automáticamente los scripts SQL sobre Cloud SQL. Los cambios de tablas, datos iniciales o privilegios de producción requieren un procedimiento específico, independiente del despliegue de las imágenes.

Cuando las nuevas revisiones están preparadas y reciben tráfico, el backend actualizado atiende las solicitudes de la aplicación móvil codigo-backend,cloudrun-iam.

Funcionamiento de la aplicación móvil



codigo-app,codigo-backend,google-id,cloudsql,cloudrun-iam

Cuando el usuario abre la aplicación instalada, Android inicia sus componentes nativos y React Native presenta la interfaz. Hermes ejecuta el código del cliente incorporado en el APK. La app intenta recuperar una sesión Google anterior y, si la encuentra, solicita el catálogo y el perfil mediante GET /v1/catalog y GET /v1/me. La versión 6.1.1 coordina esas peticiones para compartir una única operación nativa getTokens en curso, evitando que dos llamadas simultáneas sobrescriban sus promesas. La operación se libera al terminar o fallar, permitiendo reintentos y nuevas obtenciones de tokens codigo-app,hermes.

La aplicación presenta una pantalla de acceso con Google antes de cargar los datos protegidos. Una vez autenticado el usuario, la Store muestra los modelos activos del catálogo y *Mis modelos* filtra los IDs que el backend comunica como habilitados. El menú agrupa las fichas por área. Cada modelo muestra nombre, descripción, ID, fórmula y tres campos numéricos. El cliente comprueba las entradas para informar de errores al usuario; el cálculo y la autorización se resuelven en el servidor. Un fallo al cargar los datos se distingue de una lista realmente vacía o de la ausencia de permisos codigo-app,codigo-backend.

La autenticación utiliza la integración nativa de Google Sign-In. El cliente Android se identifica mediante el paquete com.storeml.app y la huella del certificado de firma registrados en Google. El ID del cliente OAuth de tipo Web configura la audiencia esperada por el backend. La app obtiene un ID token y lo incluye en el encabezado Authorization: Bearer de las peticiones HTTPS. El catálogo, el perfil, las solicitudes de acceso y los cálculos utilizan este mecanismo codigo-app,google-id.

Antes de atender cada ruta, FastAPI verifica el token con google-auth, comprueba la audiencia, el emisor, la vigencia y que se trate de un Gmail verificado. El identificador estable es el sub de Google, almacenado como google_sub en usuarios. El Gmail se obtiene del token validado, se normaliza y se registra junto con esa identidad. La API no toma la identidad de un correo arbitrario enviado en el cuerpo de la petición. Los permisos reservados previamente por Gmail se vinculan al subject verificado durante el registro codigo-backend,google-id.

Después de verificar la identidad, la API consulta PostgreSQL. La tabla modelos contiene el catálogo, y permisos relaciona cada Gmail con un ID de modelo, su estado de habilitación y la identidad vinculada. /v1/me devuelve el correo verificado, los IDs autorizados y las solicitudes pendientes. La carga inicial prepara 24 fichas y reserva doce accesos para la cuenta indicada en el SQL: KS, CART y Red Neuronal en cada una de las cuatro áreas. Los permisos efectivos dependen de los registros presentes en la base de datos codigo-backend.

Cuando se solicita un modelo, React Native envía POST /v1/requests con su modelId. FastAPI comprueba que exista y esté activo, y guarda una fila en solicitudes asociada al usuario autenticado, con estado pending. La combinación usuario/modelo es única, por lo que repetir una solicitud no crea duplicados. El administrador puede revisarlas en Cloud SQL Studio. Para conceder acceso debe crear o habilitar el registro de permisos; cambiar únicamente el estado de la solicitud a approved no otorga autorización. La app consulta los cambios al actualizar sus datos codigo-app,codigo-backend.

Al utilizar una calculadora, el cliente envía las entradas a POST /v1/models/{ID}/predict. La API comprueba el permiso, el estado del modelo, los campos y los rangos, y aplica una cuota de 30 cálculos por minuto y usuario. Después llama al motor privado con un ID token de la cuenta de servicio. El motor selecciona un módulo Python permitido por su registro fijo y ejecuta su función calcular. Por ejemplo, el ID 10000002 corresponde a inference/modelos/inmobiliario/cart.py. En esta versión, la función suma los tres inputs. La API verifica nuevamente el acceso antes de entregar el resultado, y React actualiza su presentación codigo-app,codigo-backend,cloudrun-iam.

Finalmente, el sistema separa las credenciales y responsabilidades. La contraseña PostgreSQL permanece en Secret Manager y se suministra a la API; el teléfono no recibe credenciales SQL ni claves de cuentas de servicio. La API utiliza Cloud SQL Client, el conector Python de Cloud SQL y un usuario PostgreSQL con permisos limitados. El motor no accede a la base y requiere autenticación IAM desde la API. Las consultas SQL son

parametrizadas, los permisos se comprueban en el servidor y la columna archivo documenta una ruta del repositorio sin autorizar la importación de código arbitrario. El cliente, la autenticación, el cálculo y el almacenamiento quedan así distribuidos entre componentes con funciones diferentes codigo-backend,cloudsql,cloudrun-iam.

Estructura del Repositorio de GitHub

La estructura siguiente reúne los componentes del repositorio App-Tutorial-II y, dentro de mobile, los archivos de la versión ligera 6.1.1 entregada. El ZIP móvil contiene únicamente esa carpeta y su documentación; los directorios del backend permanecen en el proyecto existente. Esta representación no implica que el ZIP o sus cambios móviles se hayan publicado en main codigo-app,codigo-backend.

```
App-Tutorial-II/                        # Proyecto existente
|-- .github/workflows/deploy.yml        # Pruebas y despliegue backend
|-- api/
|   |-- Dockerfile                      # Imagen Python de la API
|   |-- requirements.txt                # Dependencias del servidor
|   |-- requirements-test.txt           # Dependencias de pruebas
|   |-- tests/                          # API e integracion PostgreSQL
|   `-- automind_api/
|       |-- main.py                     # Inicio de la API
|       |-- app.py                      # Rutas, validacion y permisos
|       |-- auth.py                     # Verificacion del token Google
|       |-- database.py                 # Conexion con Cloud SQL
|       |-- repository.py               # Consultas SQL parametrizadas
|       `-- predictor.py                # Invocacion del motor privado
|-- inference/
|   |-- Dockerfile                      # Imagen del motor Python
|   |-- app.py                          # Endpoint privado /predict
|   |-- engine.py                       # Despacho y validacion
|   |-- registry.py                     # IDs y modulos permitidos
|   |-- catalog.json                    # Metadatos de los 24 modelos
|   |-- test_models.py                  # Pruebas de las calculadoras
|   `-- modelos/
|       |-- inmobiliario/               # Seis archivos .py
|       |-- telecomunicaciones/         # Seis archivos .py
|       |-- mineria/                    # Seis archivos .py
|       `-- forestal/                   # Seis archivos .py
|-- sql/
|   |-- 01-esquema.sql                  # Cuatro tablas de aplicacion
|   |-- 02-catalogo-y-accesos.sql        # Catalogo y doce accesos
|   `-- 03-privilegios-app.sql           # Rol SQL limitado
|-- catalog/                            # Catalogo y accesos iniciales
|-- infra/github_config.py              # Variables del despliegue
|-- docs/                               # Guia de Google Cloud
`-- mobile/                             # Cliente ligero entregado
    |-- App.tsx                         # Login, Store y calculadoras
    |-- package.json                    # Dependencias y comandos
    |-- package-lock.json               # Versiones resueltas
    |-- tsconfig.json                   # Tipos TypeScript
    |-- metro.config.js                 # Empaquetado JavaScript
    |-- src/
    |   |-- Retro.tsx                   # Fuente, papel y elementos UI
    |   |-- api.ts                      # HTTPS y tokens coordinados
    |   `-- config.ts                   # URL API y cliente OAuth Web
    |-- scripts/
```

```
|   |-- build-apk.cjs              # Compilacion local del APK
|   `-- test-api.cjs               # Regresion de tokens Google
|-- assets/fonts/                  # Fuente y licencia
|-- android/
|   |-- build.gradle               # SDK, Kotlin y JDK
|   |-- gradle.properties          # ABI, Hermes y caches
|   |-- gradlew.bat                # Wrapper Gradle para Windows
|   |-- gradle/                    # Wrapper y seleccion JDK 17
|   `-- app/
|       |-- build.gradle           # Paquete, versiones y firma
|       |-- debug.keystore         # Firma de prueba existente
|       `-- src/main/              # Manifiesto, Kotlin y assets
`-- ios/                           # Proyecto nativo para Xcode
```

codigo-app,codigo-backend.

Principales Herramientas de JavaScript utilizadas

- **Node.js:** Entorno de ejecución de JavaScript utilizado en el equipo de desarrollo para ejecutar npm, Metro y los scripts del proyecto. Permite instalar las dependencias con npm ci, ejecutar las pruebas del cliente y lanzar el script que inicia Gradle. En esta arquitectura, Node.js forma parte de las herramientas de construcción del frontend; el servidor de la aplicación está escrito en Python node,codigo-app.

- **React.js:** Biblioteca para construir interfaces a partir de componentes y estado. En la app organiza el acceso con Google, las listas de modelos, la selección de áreas, los campos y los resultados. Cuando cambia el estado, React actualiza los componentes correspondientes. La representación visual se realiza mediante React Native y los controles de la plataforma móvil react,react-native,codigo-app.

- **React Native:** Framework basado en React que permite desarrollar la interfaz móvil con JavaScript y TypeScript y utilizar componentes nativos. En la versión ligera, Text y View presentan los textos, líneas y elementos visuales; Google Sign-In y Safe Area Context proporcionan las integraciones nativas necesarias. El proyecto Android se construye con Gradle y genera un APK instalable. La presentación conserva la fuente de máquina de escribir sin requerir Skia o RaTeX react-native,codigo-app.

En el proyecto, TypeScript añade tipos al código de la interfaz y tsc -noEmit comprueba su consistencia. Metro transforma y empaqueta el código del cliente, mientras que Hermes permite ejecutar el bundle compilado en Android. Gradle integra ese código, las librerías y los recursos nativos para producir el APK. La versión 6.1.1 compila ARM64 por defecto, utiliza cachés de tareas y configuración y permite desarrollar con Fast Refresh en la variante de depuración. El APK preview incorpora el bundle y funciona sin un servidor Metro. Por su parte, Docker prepara las imágenes Python que Cloud Run ejecuta para atender las solicitudes y consultar PostgreSQL. De esta manera, Node.js participa en las herramientas de

desarrollo y construcción, React Native y Hermes intervienen en el cliente instalado y Python realiza la autenticación del lado del servidor, la autorización, el acceso a datos y la ejecución de las calculadoras.

Tutorial

Alcance y arquitectura

El proyecto utiliza una aplicación móvil React Native escrita en TypeScript, una API FastAPI, un motor Python separado y una base de datos PostgreSQL.

Nombres e identificadores de referencia

Elemento	Valor
Repositorio GitHub	artemioadaysolvers/App-Tutorial-II
Rama	main
Proyecto visible	MLStore
ID del proyecto	mlstore-513620
Número de proyecto	286807177004
Región	southamerica-west1, Santiago
Cuenta de API	storeml-api
Cuenta del motor	storeml-inference
Cuenta de despliegue	storeml-deploy
Grupo WIF / proveedor	github / storeml
Artifact Registry	storeml, formato Docker
Instancia Cloud SQL	storeml-pg
Base PostgreSQL	storeml_database
Usuario PostgreSQL	storeml_app
Secreto	storeml-db-password
Servicio API	automind-api
Servicio del motor	automind-inference
Paquete Android	com.storeml.app
Bundle ID iOS	com.automind
Correo de la cuenta de despliegue	NOMBRE_CUENTA@ID_PROYECTO.iam.gserviceaccount.com
ID numérico del propietario GitHub	ID_PROPIETARIO_GITHUB

Descargar la Aplicación Web y subir el Proyecto a GitHub

Aplicación Web: artemioadaysolvers/Tutorial-App-II.

Para descargar la aplicación móvil y subir sus archivos a un repositorio propio de GitHub, se utilizará **GitHub Desktop**, una aplicación que permite trabajar con repositorios Git mediante una interfaz gráfica, sin necesidad de ejecutar comandos desde una terminal

1. Descargar e instalar **GitHub Desktop** desde https://desktop.github.com/. Una vez instalada la aplicación, iniciar sesión con una cuenta de GitHub.
2. Crear un repositorio para almacenar el proyecto. Existen dos alternativas:
   - **Crear un repositorio desde GitHub Desktop:** seleccionar **File** → **New Repository**, indicar el nombre del repositorio y su ubicación local. Posteriormente, seleccionar **Publish repository** para publicarlo en GitHub.
   - **Clonar un repositorio creado previamente desde la página web de GitHub:** si el repositorio ya fue creado manualmente en GitHub, seleccionar **File** → **Clone Repository**, elegir el repositorio vacío y presionar **Clone**. Esto creará una copia local del repositorio en el computador.
3. En GitHub Desktop, seleccionar el repositorio de destino y abrir **Repository** → **Show in Explorer**. Esto abrirá en el Explorador de archivos de Windows la carpeta local asociada al repositorio.
4. Acceder al repositorio de la aplicación web: artemioadaysolvers/Tutorial-App-II. Seleccionar **Code** → **Local** → **Download ZIP** para descargar el proyecto completo en un archivo comprimido.
5. Descomprimir el archivo ZIP descargado. A continuación, copiar todos los archivos y carpetas contenidos en el proyecto y pegarlos directamente en la carpeta local del repositorio abierta anteriormente mediante **Show in Explorer**. Se debe conservar la estructura original del proyecto y evitar introducir una carpeta adicional que contenga

todos sus archivos.

6. Regresar a **GitHub Desktop**. En la pestaña **Changes** aparecerán automáticamente los archivos incorporados al repositorio. En el campo **Summary**, escribir una descripción de los cambios, por ejemplo, *Carga inicial de la aplicación web*. Posteriormente, presionar **Commit to main** para registrar los cambios en el repositorio local.
7. Finalmente, seleccionar **Push origin** para enviar los cambios registrados al repositorio remoto alojado en GitHub. Una vez finalizada la operación, abrir el repositorio desde la página web de GitHub y comprobar que todos los archivos y carpetas se encuentren correctamente publicados.

Es importante distinguir entre **Commit to main**, que registra los cambios en el repositorio local del computador, y **Push origin**, que envía dichos cambios al repositorio remoto de GitHub. Por el contrario, **Pull origin** permite incorporar al repositorio local los cambios disponibles en el repositorio remoto.

Crear el repositorio

1. Entrar en GitHub y seleccionar **New repository**.
2. Elegir propietario y nombre, por ejemplo Tutorial-App-II.
3. Elegir **Private** (Azure deberá recibir autorización para acceder al repositorio mediante la conexión con GitHub).
4. No agregar README, licencia ni .gitignore desde el asistente: el proyecto ya contiene los archivos necesarios.
5. Pulsar **Create repository** y copiar su URL real.

Subir los archivos con GitHub Desktop

1. Instalar GitHub Desktop e iniciar sesión.
2. Abrir **File** → **Clone repository**, elegir el repositorio o pegar su URL, seleccionar una carpeta local y pulsar **Clone**.
3. Abrir **Repository** → **Show in Explorer**.
4. Copiar el contenido del proyecto descomprimido dentro de esa carpeta. Conservar la carpeta .git administrada por GitHub Desktop.
5. Revisar **Changes**: deben aparecer el Dockerfile, el backend y las carpetas de la aplicación.
6. Comprobar que la rama se llame main; si no, renombrarla desde las opciones de rama antes de conectar Azure.
7. Escribir un resumen y pulsar **Commit to main**.
8. Pulsar **Push origin** o **Publish branch**, según corresponda.
9. Abrir GitHub en el navegador y comprobar que el Dockerfile está en la primera lista de archivos.

Consultar los identificadores de GitHub

1. Abrir **Actions**.
2. Seleccionar **AutoMind - pruebas y despliegue**.
3. Pulsar **Run workflow** sobre main.
4. Activar solo_identidad.
5. Abrir el job identidad.
6. Abrir DatosdelrepositorioparaWorkloadIdentityFederation.
7. Anotar REPOSITORY_ID y OWNER_ID.

Crear el proyecto y activar la facturación

1. Abrir https://console.cloud.google.com/.
2. En el selector superior, elegir **Nuevo proyecto**.
3. Escribir MLStore como nombre visible.
4. Crear el proyecto y copiar su ID y número.
5. Abrir **Facturación** y vincular una cuenta activa.
6. Crear un presupuesto con alertas de consumo.

Las alertas de presupuesto no establecen automáticamente un límite de gasto.

Cloud Run puede utilizar un mínimo de cero instancias. Cloud SQL mantiene cargos asociados a su instancia y almacenamiento aunque la API tenga poco tráfico.

Habilitar las API de Google Cloud

En **APIs y servicios** → **Biblioteca**, habilitar:

- Cloud Run Admin API
- Artifact Registry API
- Cloud SQL Admin API
- Secret Manager API
- Identity and Access Management API
- IAM Service Account Credentials API
- Security Token Service API
- Cloud Resource Manager API

Crear las cuentas de servicio

En **IAM y administración** → **Cuentas de servicio**, crear las siguientes identidades.

Cuenta de ejecución de la API Crear storeml-api y concederle el siguiente rol:

- Cloud SQL Client: roles/cloudsql.client.

Cuenta de ejecución del motor Crear storeml-inference.

Cuenta de despliegue Crear storeml-deploy y concederle los siguientes roles:

- Cloud Run Developer: roles/run.developer.
- Service Usage Consumer: roles/serviceusage.serviceUsageConsumer.

Seleccionar storeml-api y abrir **Permisos** → **Principales con acceso** → **Otorgar acceso**.

En **Entidades nuevas**, ingresar el correo completo de la cuenta de despliegue NOMBRE_CUENTA@ID_PROYECTO.iam.gserviceaccount.com y asignar Rol Usuariodecuentadeservicio.

Repetir el procedimiento sobre la cuenta del motor storeml-inference, utilizando el mismo correo de la cuenta de despliegue y el mismo rol.

Vincular GitHub con la cuenta de servicio

1. Ingresar a Google Cloud Console y dirigirse a **IAM y Administración** → **Federación de identidades para cargas de trabajo**.
2. Localizar el grupo de identidades configurado para GitHub y seleccionar **Otorgar acceso**.
3. Seleccionar la opción **Otorgar acceso con identidad temporal como cuenta de servicio**.
4. En el apartado **Cuenta de servicio**, seleccionar:

   storeml-deploy

5. En el apartado **Selecciona principales**, correspondiente a las identidades que pueden acceder a la cuenta de servicio, establecer la siguiente configuración:
   - **Nombre del atributo:** repository_id.
   - **Valor del atributo:** 1421519665.
6. Verificar que el identificador del repositorio de GitHub corresponda al valor configurado.
7. Seleccionar **Guardar** para conceder el acceso a la cuenta de servicio.

Esta configuración permite que los workflows de **GitHub Actions** del repositorio autorizado utilicen la cuenta de servicio storeml-deploy mediante **Workload Identity Federation**, sin necesidad de almacenar claves privadas de cuentas de servicio en GitHub.

El acceso queda restringido a las identidades que cumplan la condición del atributo repository_id, siempre que dicho atributo esté correctamente configurado en el proveedor de identidades.

Configurar las variables de autenticación en GitHub

1. Ingresar a **GitHub** y abrir el repositorio correspondiente al proyecto.
2. En la barra superior del repositorio, seleccionar **Settings**.
3. En el menú lateral, dirigirse a **Secrets and variables** → **Actions**.
4. Seleccionar la pestaña **Variables**.
5. Pulsar **New repository variable** para crear la primera variable.
6. En el campo **Name**, ingresar:

   GCP_PROJECT_ID

   En el campo **Value**, ingresar:

   mlstore-513620

7. Seleccionar **Add variable** para guardar la configuración.
8. Nuevamente, seleccionar **New repository variable** para crear la segunda variable.
9. En el campo **Name**, ingresar:

   GCP_DEPLOY_SERVICE_ACCOUNT

   En el campo **Value**, ingresar:

   storeml-deploy@mlstore-513620.iam.gserviceaccount.com

10. Seleccionar **Add variable** para guardar la configuración.
11. Nuevamente, seleccionar **New repository variable** para crear la tercera variable.
12. En el campo **Name**, ingresar:

    GCP_WIF_PROVIDER

    En el campo **Value**, ingresar:

    projects/286807177004/locations/global/workloadIdentityPools/github/providers/storeml

13. Seleccionar **Add variable** para guardar la configuración.

14. Verificar que las tres variables aparezcan en el listado de **Repository variables** con sus respectivos valores.

Estas variables permiten que el workflow de **GitHub Actions** identifique el proyecto de Google Cloud, la cuenta de servicio utilizada para el despliegue y el proveedor de identidades configurado mediante **Workload Identity Federation**.

De esta manera, GitHub Actions puede solicitar credenciales temporales para autenticarse en Google Cloud, sin necesidad de almacenar claves privadas de cuentas de servicio en el repositorio.

Crear el repositorio en Artifact Registry

1. Ingresar a **Google Cloud Console** y seleccionar el proyecto MLStore (mlstore-513620).
2. Dirigirse al servicio **Artifact Registry** → **Repositorios**.
3. Seleccionar **Crear repositorio** para configurar el almacenamiento de las imágenes Docker del backend.
4. En el apartado **Nombre**, ingresar:

   storeml

5. En el apartado **Formato**, seleccionar Docker.
6. En el apartado **Modo**, seleccionar EstÃąndar.
7. En el apartado **Tipo de ubicación**, seleccionar RegiÃşn.
8. En el apartado **Región**, seleccionar Santiago(southamerica-west1).
9. En el apartado **Cifrado**, conservar la opción **Clave administrada por Google**.
10. En el apartado **Análisis de vulnerabilidades**, seleccionar **Inhabilitado**.

    Importante: Mantener deshabilitado el análisis de vulnerabilidades para evitar cargos adicionales asociados a este servicio. Esta configuración es especialmente importante en instalaciones de demostración con presupuesto limitado.
11. Verificar que todos los parámetros correspondan a la configuración indicada, especialmente que el análisis de vulnerabilidades permanezca **inhabilitado**.
12. Seleccionar **Crear repositorio** para finalizar el procedimiento.

El repositorio storeml permitirá almacenar las imágenes Docker de los componentes api e inference del backend.

Estas imágenes podrán publicarse mediante **GitHub Actions** y utilizarse posteriormente para desplegar los servicios en **Google Cloud Run**.

Nota: Deshabilitar el análisis de vulnerabilidades evita los cargos asociados a dicha funcionalidad, pero no elimina los posibles costos de almacenamiento y transferencia de datos de Artifact Registry.

Configurar los permisos de Artifact Registry

1. Ingresar a **Google Cloud Console** y seleccionar el proyecto MLStore (mlstore-513620).
2. Dirigirse al servicio **Artifact Registry** → **Repositorios**.
3. Localizar el repositorio storeml y marcar la casilla ubicada a su izquierda.
4. En la barra superior, seleccionar **Mostrar panel de información**.
5. En el panel lateral, abrir la pestaña **Permisos**.
6. Seleccionar **Agregar principal** u **Otorgar acceso**, según la opción disponible en la consola.
7. En el apartado **Agregar entidades**, ingresar la siguiente cuenta de servicio:

   storeml-deploy@mlstore-513620.iam.gserviceaccount.com

8. En el apartado **Asignar roles**, seleccionar **Seleccionar un rol**.
9. Buscar y seleccionar el rol **Escritor de Artifact Registry** (*Artifact Registry Writer*), correspondiente a:

   roles/artifactregistry.writer

10. Verificar que la cuenta de servicio storeml-deploy tenga asignado el rol correspondiente.
11. Seleccionar **Guardar** para conceder los permisos.

Esta configuración permite que la cuenta de servicio storeml-deploy publique y administre imágenes Docker dentro del repositorio storeml.

De esta manera, los workflows de **GitHub Actions** podrán autenticarse mediante **Workload Identity Federation** y publicar las imágenes de los componentes api e inference en **Artifact Registry**.

Importante: El rol roles/artifactregistry.writer debe concederse únicamente sobre el repositorio storeml, evitando otorgar permisos de escritura innecesarios sobre otros repositorios del proyecto.

Configurar las variables de Artifact Registry en GitHub

1. Ingresar a **GitHub** y abrir el repositorio correspondiente al proyecto.
2. En la barra superior del repositorio, seleccionar **Settings**.
3. En el menú lateral, dirigirse a **Secrets and variables** → **Actions**.
4. Seleccionar la pestaña **Variables**.
5. Pulsar **New repository variable** para crear la primera variable.
6. En el campo **Name**, ingresar:

   ARTIFACT_REPOSITORY

   En el campo **Value**, ingresar:

   storeml
7. Seleccionar **Add variable** para guardar la configuración.
8. Nuevamente, seleccionar **New repository variable** para crear la segunda variable.
9. En el campo **Name**, ingresar:

   GCP_REGION

   En el campo **Value**, ingresar:

   southamerica-west1
10. Seleccionar **Add variable** para guardar la configuración.
11. Verificar que ambas variables aparezcan en el listado de **Repository variables** con sus respectivos valores.

Estas variables permiten que los workflows de **GitHub Actions** identifiquen automáticamente el repositorio storeml de **Artifact Registry** y la región southamerica-west1 de Google Cloud.

De esta manera, el proceso de integración y despliegue continuo podrá publicar las imágenes Docker de los componentes api e inference en la ubicación correspondiente, sin necesidad de incorporar estos valores directamente en el código del workflow.

Crear una instancia de PostgreSQL en Cloud SQL

1. Ingresar a **Google Cloud Console** y seleccionar el proyecto MLStore (mlstore-513620).
2. Dirigirse al servicio **Cloud SQL**.
3. Desplazarse hasta la sección **PostgreSQL** y seleccionar **Zona de pruebas**.
4. En el apartado **Edición**, conservar la opción Enterprise.
5. En el apartado **Ajuste predeterminado de edición**, mantener seleccionada la opción Zonadepruebas.
6. En el apartado **Versión de la base de datos**, cambiar PostgreSQL18 por PostgreSQL17.
7. En el apartado **ID de instancia**, ingresar:

   storeml-pg

8. Crear una contraseña segura para el usuario administrador postgres y almacenarla de forma privada.
9. En el apartado **Región**, seleccionar Santiago(southamerica-west1), correspondiente a la región utilizada por el repositorio de Artifact Registry.
10. En el apartado **Disponibilidad**, conservar la configuración ZonaÃžnica.
11. Desplegar el apartado **Personaliza tu instancia**.
12. Seleccionar la sección **Configuración de la máquina, CPU y memoria**.
13. En el apartado **Tipo de máquina**, seleccionar **Uso general** y posteriormente **Núcleo compartido**.
14. Seleccionar el tipo de máquina db-f1-micro, cuya configuración corresponde a:
    - **CPU:** 1 CPU virtual compartida.
    - **Memoria RAM:** 0,614 GB.
15. Verificar que la configuración seleccionada corresponda al tipo de máquina db-f1-micro y que la consola muestre los recursos esperados.

Crear la base de datos y el usuario en Cloud SQL

Crear la base de datos PostgreSQL

1. Ingresar a **Google Cloud Console** y seleccionar el proyecto MLStore (mlstore-513620).
2. Dirigirse al servicio **Cloud SQL**.
3. Localizar la instancia storeml-pg y seleccionarla.
4. Esperar hasta que la instancia indique que se encuentra **Disponible**.
5. En el menú lateral de la instancia, seleccionar **Bases de datos**.
6. Seleccionar **Crear base de datos**.
7. En el apartado **Nombre de la base de datos**, ingresar:

   storeml_database

8. Seleccionar **Crear** para confirmar la creación de la base de datos.
9. Verificar que storeml_database aparezca en el listado de bases de datos de la instancia storeml-pg.

Crear el usuario de la aplicación

1. Regresar al servicio **Cloud SQL** de Google Cloud Console.
2. Seleccionar la instancia storeml-pg y verificar que se encuentre **Disponible**.
3. En el menú lateral de la instancia, seleccionar **Usuarios**.
4. Seleccionar **Agregar cuenta de usuario**.
5. En el apartado **Nombre de usuario**, ingresar:

   storeml_app

6. En el apartado **Contraseña**, generar una contraseña segura para el nuevo usuario.
7. Guardar la contraseña en un lugar privado y seguro, ya que será necesaria para configurar la conexión del backend con PostgreSQL.
8. Seleccionar **Crear** o **Agregar** para confirmar la creación del usuario.
9. Verificar que el usuario storeml_app aparezca en el listado de usuarios de la instancia storeml-pg.

Esta configuración permite separar las credenciales del administrador postgres de las credenciales utilizadas por la aplicación.

La base de datos storeml_database almacenará la información persistente del proyecto, mientras que el usuario storeml_app se utilizará para establecer las conexiones del backend con **PostgreSQL**.

Importante: La contraseña del usuario storeml_app no debe incorporarse directamente en el código fuente ni almacenarse en el repositorio de GitHub. Se recomienda utilizar **Google Cloud Secret Manager** para administrar las credenciales de forma segura.

Asimismo, se deben verificar los permisos del usuario sobre la base de datos storeml_database antes de utilizarlo desde la aplicación.

Conectar PostgreSQL mediante Cloud SQL Studio

1. Ingresar a **Google Cloud Console** y seleccionar el proyecto MLStore (mlstore-513620).
2. Dirigirse al servicio **Cloud SQL**.
3. Localizar la instancia storeml-pg y seleccionarla.
4. Esperar hasta que la instancia indique que se encuentra **Disponible**.
5. En el menú lateral de la instancia, seleccionar **Cloud SQL Studio**.
6. En la pantalla de conexión, completar los siguientes campos:
   - **Base de datos:** storeml_database.
   - **Autenticación:** Integrada.
   - **Usuario:** postgres.
   - **Contraseña:** ingresar la contraseña del usuario administrador postgres, almacenada durante la creación de la instancia.
7. Verificar que los datos de conexión correspondan a la instancia y base de datos configuradas anteriormente.
8. Seleccionar **Autenticar** o **Conectar**, según la opción disponible en la consola.
9. Esperar hasta que **Cloud SQL Studio** establezca correctamente la conexión con PostgreSQL.
10. Verificar que se encuentre disponible el editor de consultas SQL, desde el cual será posible ejecutar instrucciones para crear y administrar las tablas de storeml_database.

Esta configuración permite acceder directamente a la base de datos storeml_database mediante **Cloud SQL Studio**, utilizando las credenciales del administrador postgres.

El editor SQL permitirá ejecutar instrucciones **DDL** (*Data Definition Language*) para crear las tablas, definir sus columnas, establecer claves primarias y configurar las relaciones necesarias para el funcionamiento del backend de la aplicación.

Importante: La cuenta postgres se utilizará únicamente para las tareas administrativas y de configuración inicial. Para las operaciones habituales del backend se recomienda utilizar el usuario storeml_app, con los permisos estrictamente necesarios sobre la base de datos.

Crear las tablas y configurar los permisos en PostgreSQL

1. En **Cloud SQL Studio**, verificar que se encuentre seleccionada la base de datos:

   storeml_database

2. Seleccionar **Nueva pestaña del editor de SQL** para abrir una nueva consulta.
3. Ingresar al repositorio de GitHub y abrir el archivo **01-esquema.sql**, disponible en:

   https://github.com/artemioadaysolvers/App-Tutorial-II/blob/main/sql/01-esquema.sql

4. Copiar el contenido completo del archivo 01-esquema.sql y pegarlo en el editor de consultas de **Cloud SQL Studio**.
5. Seleccionar **Ejecutar** y verificar que las instrucciones SQL se completen sin errores.
6. Nuevamente, comprobar que se encuentre seleccionada la base de datos storeml_database y abrir una **Nueva pestaña del editor de SQL**.
7. Ingresar al repositorio de GitHub y abrir el archivo **02-catalogo-y-accesos.sql**, disponible en:

   https://github.com/artemioadaysolvers/App-Tutorial-II/blob/main/sql/02-catalogo-y-accesos.sql

8. Copiar el contenido completo del archivo 02-catalogo-y-accesos.sql y pegarlo en la pestaña **Consulta sin título** de Cloud SQL Studio.
9. Seleccionar **Ejecutar** y verificar que las instrucciones SQL se completen sin errores.
10. Nuevamente, comprobar que se encuentre seleccionada la base de datos storeml_database y abrir una **Nueva pestaña del editor de SQL**.
11. Ingresar al repositorio de GitHub y abrir el archivo **03-privilegios-app.sql**, disponible en:

    https://github.com/artemioadaysolvers/App-Tutorial-II/blob/main/sql/03-privilegios-app.sql

12. Copiar el contenido completo del archivo 03-privilegios-app.sql y pegarlo en el editor de consultas de **Cloud SQL Studio**.

13. Seleccionar **Ejecutar** y verificar que las instrucciones SQL se completen sin errores.
14. Para comprobar que el usuario storeml_app no conserve privilegios administrativos, abrir una

    Nueva pestaña del editor de SQL.
15. Copiar y ejecutar la siguiente consulta:

```
SELECT
  rolcreatedb,
  rolcreaterole,
  pg_has_role(
    'storeml_app',
    'cloudsqlsuperuser',
    'MEMBER'
  ) AS tiene_cloudsqlsuperuser
FROM pg_roles
WHERE rolname = 'storeml_app';
```

16. Seleccionar **Ejecutar** y revisar los resultados de la consulta.
17. Verificar que los tres valores obtenidos sean FALSE:
    - **rolcreatedb:** FALSE.
    - **rolcreaterole:** FALSE.
    - **tiene_cloudsqlsuperuser:** FALSE.
18. Confirmar que el usuario storeml_app no posea los privilegios administrativos evaluados antes de continuar con la configuración del backend.

La ejecución de estos tres archivos SQL permite preparar la estructura de la base de datos, configurar el catálogo y los accesos de la aplicación, y establecer los privilegios correspondientes al usuario storeml_app.

Importante: Los tres resultados de la consulta de verificación deben ser FALSE. Esto confirma que el usuario storeml_app no tiene permisos para crear bases de datos, crear roles ni pertenecer al rol administrativo cloudsqlsuperuser.

Esta verificación contribuye a aplicar el **principio de mínimo privilegio**, reduciendo los permisos administrativos disponibles para la cuenta utilizada por el backend.

Si alguno de los resultados es TRUE, se deberá revisar la configuración de roles y privilegios antes de continuar.

Nota: Esta consulta verifica únicamente los tres privilegios indicados. No comprueba por sí sola todos los permisos del usuario sobre las tablas, secuencias y esquemas de PostgreSQL.

Almacenar la contraseña de PostgreSQL en Secret Manager

1. Ingresar a **Google Cloud Console** y seleccionar el proyecto MLStore (mlstore-513620).
2. Dirigirse a **Seguridad** → **Secret Manager**.

   También es posible acceder directamente mediante:

   https://console.cloud.google.com/security/secret-manager/create?project=mlstore-513620

3. Seleccionar **Crear secreto** (*Create secret*).
4. En el apartado **Nombre**, ingresar:

   storeml-db-password

5. En el apartado **Tipo** (*Type*), seleccionar:

   Other

6. En el apartado **Valor del secreto**, ingresar la contraseña exacta del usuario de PostgreSQL:

   storeml_app

   La contraseña debe introducirse exactamente como fue generada durante la creación del usuario, **sin comillas ni espacios adicionales**.
7. Revisar las opciones de configuración del secreto y los posibles costos asociados a su almacenamiento y acceso.
8. Seleccionar **Crear secreto** para guardar la contraseña de forma segura.
9. Verificar que el secreto storeml-db-password aparezca en el listado de **Secret Manager** y que disponga de una versión habilitada.

Esta configuración permite almacenar de forma segura la contraseña del usuario storeml_app mediante **Google Cloud Secret Manager**, evitando incorporarla directamente en el código fuente del backend o en el repositorio de GitHub.

Posteriormente, el servicio desplegado en **Google Cloud Run** podrá acceder al secreto mediante una cuenta de servicio autorizada, utilizando los permisos correspondientes de **Secret Manager Secret Accessor** (roles/secretmanager.secretAccessor).

Importante: La creación del secreto no concede automáticamente permisos de lectura a Cloud Run. Estos deberán configurarse posteriormente para la cuenta de servicio utilizada por el backend, preferentemente sobre el secreto específico storeml-db-password.

Asimismo, Secret Manager puede generar cargos por versiones de secretos almacenadas y operaciones de acceso, por lo que se recomienda revisar las tarifas del servicio.

Configurar los permisos de acceso a Secret Manager

1. Ingresar a **Google Cloud Console** y seleccionar el proyecto MLStore (mlstore-513620).
2. Dirigirse a **Seguridad** → **Secret Manager**.
   También es posible acceder directamente mediante:

   https://console.cloud.google.com/security/secret-manager?project=mlstore-513620

3. Localizar el secreto storeml-db-password y marcar la casilla ubicada a su izquierda.
4. En la barra superior, seleccionar **Mostrar panel de información**.
5. En el panel lateral, abrir la pestaña **Permisos**.
6. Seleccionar **Agregar principal** u **Otorgar acceso**, según la opción disponible en la consola.
7. En el apartado **Agregar entidades**, ingresar la siguiente cuenta de servicio:

   storeml-api@mlstore-513620.iam.gserviceaccount.com

8. En el apartado **Asignar roles**, seleccionar **Seleccionar un rol**.
9. Buscar y seleccionar el rol **Usuario con acceso a secretos de Secret Manager** (*Secret Manager Secret Accessor*), correspondiente a:

   roles/secretmanager.secretAccessor

10. Verificar que la cuenta de servicio storeml-api tenga asignado el rol correspondiente.
11. Seleccionar **Guardar** para conceder los permisos.
12. Confirmar que la cuenta de servicio storeml-api aparezca en el listado de principales autorizados del secreto storeml-db-password.

Esta configuración permite que la cuenta de servicio storeml-api, utilizada por el backend desplegado en **Google Cloud Run**, acceda a la contraseña del usuario storeml_app almacenada en **Google Cloud Secret Manager**.

De esta manera, la API podrá recuperar las credenciales necesarias para establecer una conexión con la base de datos storeml_database, sin

necesidad de incorporar contraseñas directamente en el código fuente ni en el repositorio de GitHub.

Importante: El rol roles/secretmanager.secretAccessor debe concederse únicamente sobre el secreto storeml-db-password, evitando otorgar permisos de lectura sobre todos los secretos del proyecto.

Esta configuración contribuye a implementar el **principio de mínimo privilegio**, limitando el acceso de la API exclusivamente a las credenciales que necesita para su funcionamiento.

Configurar las variables de conexión a PostgreSQL en GitHub

1. Ingresar a **GitHub** y abrir el repositorio correspondiente al proyecto.
2. En la barra superior del repositorio, seleccionar **Settings**.
3. En el menú lateral, dirigirse a **Secrets and variables** → **Actions**.
4. Seleccionar la pestaña **Variables**.
5. Pulsar **New repository variable** para crear la primera variable.
6. En el campo **Name**, ingresar:

   INSTANCE_CONNECTION_NAME

   En el campo **Value**, ingresar:

   mlstore-513620:southamerica-west1:storeml-pg

7. Seleccionar **Add variable** para guardar la configuración.
8. Nuevamente, seleccionar **New repository variable** para crear la segunda variable.
9. En el campo **Name**, ingresar:

   DB_PASSWORD_SECRET

   En el campo **Value**, ingresar:

   storeml-db-password

10. Seleccionar **Add variable** para guardar la configuración.
11. Nuevamente, seleccionar **New repository variable** para crear la tercera variable.
12. En el campo **Name**, ingresar:

    DB_PASSWORD_SECRET_VERSION

    En el campo **Value**, ingresar:

    1

13. Seleccionar **Add variable** para guardar la configuración.
14. Verificar que las tres variables aparezcan en el listado de **Repository variables** con sus respectivos valores.

Estas variables permiten que los workflows de **GitHub Actions** identifiquen la instancia de **Cloud SQL**, el secreto que almacena la contraseña del usuario storeml_app y la versión específica de dicho secreto.

La variable INSTANCE_CONNECTION_NAME contiene el nombre de conexión de la instancia storeml-pg, necesario para establecer la comunicación entre el backend y PostgreSQL mediante el mecanismo de conexión configurado en Google Cloud.

Por su parte, la variable DB_PASSWORD_SECRET identifica el secreto almacenado en **Google Cloud Secret Manager**, mientras que DB_PASSWORD_SECR ET_VERSION especifica la versión 1 del secreto.

Importante: La versión 1 debe existir y encontrarse habilitada en Secret Manager. Si posteriormente se modifica la contraseña del usuario st oreml_app y se crea una nueva versión del secreto, será necesario actualizar la variable DB_PASSWORD_SECRET_VERSION para utilizar la versión correspondiente.

Estas variables no contienen la contraseña de PostgreSQL, sino únicamente identificadores y referencias de configuración. Por esta razón, pueden almacenarse en **Repository variables** sin necesidad de utilizar **Repository secrets**.

Configurar Google Auth Platform y crear un cliente OAuth

Configurar la aplicación en Google Auth Platform

1. Ingresar a **Google Cloud Console** y seleccionar el proyecto MLStore (mlstore-513620).
2. Dirigirse al servicio **Google Auth Platform**.
3. Seleccionar **Comenzar** para iniciar el asistente de configuración de autenticación de Google.
4. En el apartado **Nombre de la aplicación**, ingresar:

   MLStore

5. En el apartado **Correo de asistencia al usuario**, seleccionar una dirección de correo electrónico válida para recibir consultas relacionadas con la aplicación.
6. En el apartado **Público o Audiencia**, seleccionar **Externo**, para permitir la autenticación de usuarios con cuentas de Google externas a la organización.
7. En el apartado **Información de contacto**, ingresar una dirección de correo electrónico válida para recibir notificaciones relacionadas con la configuración de OAuth.
8. Revisar la configuración y aceptar las condiciones correspondientes, si la consola lo solicita.
9. Seleccionar **Crear** para finalizar la configuración inicial de Google Auth Platform.

Crear un cliente OAuth para el backend

1. En **Google Auth Platform**, dirigirse al apartado **Clientes**.
2. Seleccionar **Crear cliente de OAuth**.
3. En el apartado **Tipo de aplicación**, seleccionar:

   AplicaciÃşnweb

4. En el apartado **Nombre**, ingresar:

   storeml-backend

5. En el apartado **Orígenes autorizados de JavaScript**, no agregar ningún origen y mantener el campo vacío.
6. En el apartado **URIs de redirección autorizados**, no agregar ninguna dirección y mantener el campo vacío.
7. Verificar que la opción **Use this client for an AI-powered agent** permanezca desmarcada.
8. Revisar que el tipo de aplicación sea AplicaciÃşnweb y que el nombre del cliente corresponda a storeml-backend.
9. Seleccionar **Crear** para generar el cliente OAuth.
10. Verificar que el cliente storeml-backend aparezca en el listado de clientes OAuth de Google Auth Platform.
11. Identificar y guardar el **ID de cliente OAuth** (*OAuth Client ID*) generado, ya que será necesario para configurar la autenticación del backend.

Esta configuración permite registrar la aplicación MLStore en **Google Auth Platform** y crear un cliente OAuth asociado al backend storeml-backend.

El identificador del cliente OAuth permitirá que el backend compruebe que los tokens de identidad de Google recibidos correspondan a la aplicación autorizada.

Configurar el ID de cliente OAuth en GitHub

1. Ingresar a **Google Cloud Console** y seleccionar el proyecto MLStore (mlstore-513620).
2. Dirigirse al servicio **Google Auth Platform** → **Clientes**.
3. Localizar el cliente OAuth storeml-backend, creado anteriormente.
4. En la columna **ID de cliente**, ubicada a la derecha de storeml-backend, copiar el identificador completo del cliente OAuth.
5. Ingresar a **GitHub** y abrir el repositorio correspondiente al proyecto.
6. En la barra superior del repositorio, seleccionar **Settings**.
7. En el menú lateral, dirigirse a **Secrets and variables** → **Actions**.
8. Seleccionar la pestaña **Variables**.
9. Pulsar **New repository variable** para crear una nueva variable.
10. En el campo **Name**, ingresar:

    GOOGLE_WEB_CLIENT_ID

11. En el campo **Value**, pegar el **ID de cliente OAuth** copiado anteriormente desde Google Auth Platform, sin comillas ni espacios adicionales.
12. Seleccionar **Add variable** para guardar la configuración.
13. Verificar que la variable GOOGLE_WEB_CLIENT_ID aparezca en el listado de **Repository variables** con el identificador correspondiente al cliente OAuth storeml-backend.

Esta configuración permite que los workflows de **GitHub Actions** utilicen el identificador del cliente OAuth registrado en **Google Auth Platform**.

La variable GOOGLE_WEB_CLIENT_ID permitirá configurar el backend para verificar los tokens de identidad de Google, comprobando que fueron emitidos para el cliente OAuth correspondiente a la aplicación.

De esta manera, el sistema podrá integrar la autenticación mediante cuentas de Google con los mecanismos de autorización implementados en el backend de MLStore.

Importante: El identificador del cliente OAuth no constituye una contraseña ni una credencial privada. Por esta razón, puede almacenarse en **Repository variables** sin necesidad de utilizar **Repository secrets**.

No obstante, el **Client Secret**, si se utiliza posteriormente, deberá almacenarse mediante un mecanismo seguro y nunca incorporarse directamente al código fuente del repositorio.

Ejecutar el primer despliegue del backend

1. Ingresar a **GitHub** y abrir el repositorio correspondiente al proyecto.
2. En la barra superior del repositorio, seleccionar **Actions**.
3. En el menú lateral, dirigirse a **All workflows** para visualizar los workflows disponibles.
4. Localizar y seleccionar el **workflow de despliegue del backend**, configurado previamente para publicar las imágenes Docker y desplegar los servicios en Google Cloud.
5. Seleccionar **Run workflow** para abrir las opciones de ejecución manual.
6. Localizar la opción:

   Mostrar IDs del repositorio para la configuración inicial, sin desplegar

   correspondiente al parámetro solo_identidad.
7. Mantener la opción solo_identidad **desmarcada**, para permitir que el workflow ejecute el proceso de despliegue del backend.
8. Verificar que se encuentre seleccionada la rama correspondiente al proyecto y que las opciones de ejecución sean correctas.
9. Seleccionar **Run workflow** para iniciar el primer despliegue del backend.
10. Esperar a que GitHub Actions inicie la ejecución del workflow y seleccionar la ejecución correspondiente para visualizar su progreso.
11. Revisar los registros de ejecución y verificar que las etapas de autenticación, construcción, publicación y despliegue se completen correctamente.
12. Confirmar que la ejecución finalice con el estado **Success**, indicado mediante una marca de verificación de color verde.

Esta ejecución permite iniciar el primer proceso de **integración y despliegue continuo (CI/CD)** del backend de MLStore, utilizando **GitHub Actions**.

Durante este procedimiento, el workflow se autenticará en Google Cloud

mediante **Workload Identity Federation** y utilizará la cuenta de servicio storeml-deploy para realizar las operaciones autorizadas.

Posteriormente, el workflow podrá construir y publicar las imágenes Docker de los componentes api e inference en **Artifact Registry** y desplegar los servicios correspondientes en **Google Cloud Run**.

Importante: La opción solo_identidad debe permanecer **desmarcada**. Si se habilita, el workflow ejecutará únicamente el procedimiento destinado a mostrar los identificadores del repositorio para la configuración inicial, sin realizar el despliegue del backend.

Asimismo, se recomienda revisar los registros de GitHub Actions y los recursos creados en Google Cloud, ya que el primer despliegue puede generar costos asociados a los servicios utilizados.

Configurar los permisos y el acceso público en Cloud Run

Permitir que la API invoque el motor privado

1. Ingresar a **Google Cloud Console** y seleccionar el proyecto MLStore (mlstore-513620).
2. Dirigirse al servicio **Cloud Run**.
3. En el menú lateral, seleccionar **Servicios** para visualizar los servicios desplegados en el proyecto.
4. Localizar el servicio automind-inference y marcar la casilla ubicada a su izquierda.
5. En la barra superior, seleccionar **Mostrar panel de información**.
6. En el panel lateral, abrir la pestaña **Permisos**.
7. Seleccionar **Agregar principal** u **Otorgar acceso**, según la opción disponible en la consola.
8. En el apartado **Entidades nuevas**, ingresar la siguiente cuenta de servicio:

   storeml-api@mlstore-513620.iam.gserviceaccount.com

9. En el apartado **Asignar roles**, seleccionar **Seleccionar un rol**.
10. Buscar y seleccionar el rol **Invocador de Cloud Run** (*Cloud Run Invoker*), correspondiente a:

    roles/run.invoker

11. Verificar que la cuenta de servicio storeml-api tenga asignado el rol correspondiente.
12. Seleccionar **Guardar** para conceder los permisos.
13. Confirmar que la cuenta de servicio storeml-api aparezca en el listado de principales autorizados del servicio automind-inference.

Esta configuración permite que la cuenta de servicio storeml-api, utilizada por el backend desplegado en **Google Cloud Run**, pueda invocar el servicio privado automind-inference.

De esta manera, la API podrá comunicarse con el motor de inferencia para solicitar predicciones de los modelos de **Machine Learning**, sin necesidad de habilitar el acceso público al motor.

Importante: El rol roles/run.invoker debe concederse únicamente sobre el servicio automind-inference. Asimismo, las solicitudes realizadas desde la API deberán incluir un token de identidad válido, emitido para la audiencia correspondiente al servicio privado.

Habilitar el acceso público a la API

1. Regresar al servicio **Cloud Run** de Google Cloud Console.
2. En el menú lateral, seleccionar **Servicios**.
3. Localizar el servicio automind-api y seleccionar su nombre para acceder a la configuración del servicio.
4. Abrir la pestaña **Seguridad**.
5. En el apartado **Autenticación**, seleccionar la opción:

   PermitiraccesopÃžblico

6. Verificar que la configuración permita recibir solicitudes HTTP desde Internet sin exigir autenticación previa mediante Cloud Run IAM.
7. Seleccionar **Ver diferencias y volver a implementar**, si la consola presenta esta opción.
8. Revisar los cambios de configuración y seleccionar **Implementar cambios** para aplicarlos al servicio.
9. Si la consola presenta directamente la opción **Guardar**, seleccionarla para confirmar los cambios.
10. Esperar hasta que Cloud Run confirme que la configuración del servicio se ha actualizado correctamente.
11. Verificar que el servicio automind-api mantenga habilitado el acceso público.

Esta configuración permite que la aplicación móvil se comunique con el servicio automind-api a través de Internet, utilizando la dirección HTTPS proporcionada por **Google Cloud Run**.

Sin embargo, permitir el acceso público al servicio no significa que cualquier usuario pueda utilizar libremente las funcionalidades del backend.

La API deberá continuar exigiendo un **token de identidad válido de Google** en las operaciones protegidas, verificando su autenticidad, audiencia y vigencia.

Posteriormente, el backend deberá comprobar los permisos asociados a cada usuario antes de autorizar el acceso a los modelos de **Machine Learning**.

De esta manera, la arquitectura separa las responsabilidades de sus componentes:

- **automind-api:** servicio accesible desde Internet, encargado de autenticar usuarios, verificar permisos y procesar las solicitudes de la aplicación móvil.
- **automind-inference:** servicio privado encargado de ejecutar los modelos de Machine Learning, accesible únicamente mediante identidades autorizadas.

Importante: La seguridad de automind-api dependerá de que el backend implemente correctamente la validación de tokens y los controles de autorización. La opción **Permitir acceso público** elimina la exigencia de autenticación de Cloud Run IAM para las solicitudes entrantes, pero no sustituye ni implementa automáticamente la autenticación de la aplicación.

Asimismo, se recomienda mantener automind-inference **sin acceso público**, para evitar que los usuarios puedan comunicarse directamente con el motor de inferencia sin pasar por los controles del backend.

Configurar usuarios de prueba y OAuth para Android

Agregar usuarios de prueba en Google Auth Platform

1. Ingresar a **Google Cloud Console** y seleccionar el proyecto MLStore (mlstore-513620).
2. Dirigirse al servicio **Google Auth Platform**.
3. En el menú lateral, seleccionar **Público** para acceder a la configuración de audiencia de la aplicación.
4. Localizar el apartado **Usuarios de prueba**.
5. Seleccionar **Agregar usuarios** para registrar una nueva cuenta de prueba.
6. Ingresar la dirección de correo electrónico de Google que se utilizará para probar la autenticación de la aplicación móvil.
7. Seleccionar **Guardar** para agregar el usuario de prueba.
8. Verificar que la dirección de correo electrónico aparezca en el listado de **Usuarios de prueba**.

Esta configuración permite que las cuentas de Google autorizadas participen en las pruebas de autenticación de MLStore, mientras la aplicación OAuth permanezca en modo de prueba.

Crear un cliente OAuth para Android

1. En **Google Auth Platform**, dirigirse al apartado **Clientes** desde el menú lateral.
2. Seleccionar **Crear cliente de OAuth** para registrar la aplicación Android.
3. En el apartado **Tipo de aplicación**, seleccionar:

   Android

4. En el apartado **Nombre**, ingresar storeml-android-debug.
5. En el apartado **Nombre del paquete**, ingresar:

   com.storeml.app

6. En el apartado **Huella digital SHA-1 del certificado**, ingresar la huella SHA-1 correspondiente al certificado utilizado para firmar la versión de desarrollo de la aplicación Android.
7. Verificar que el nombre del paquete y la huella digital SHA-1 correspondan a la aplicación Android que se ejecutará durante las pruebas.
8. Seleccionar **Crear** para generar el cliente OAuth de Android.
9. Verificar que el cliente storeml-android-debug aparezca en el listado de clientes OAuth de Google Auth Platform.

Esta configuración permite registrar la aplicación Android de MLStore en **Google Auth Platform**, asociando su nombre de paquete con el certificado digital utilizado durante el desarrollo.

El cliente OAuth storeml-android-debug permitirá identificar la aplicación Android durante el proceso de autenticación mediante cuentas de Google.

Importante: Para crear un cliente OAuth de Android, Google requiere tanto el nombre del paquete como la huella digital **SHA-1** del certificado de firma.

Esta huella debe obtenerse del certificado real utilizado por la aplicación. No debe utilizarse una huella SHA-1 de ejemplo ni una perteneciente a otro proyecto.

Asimismo, cuando la aplicación se distribuya mediante una versión de producción, será necesario registrar el certificado de firma correspondiente a dicha versión, si es diferente del utilizado durante el desarrollo.

Preparar y compilar la aplicación Android

Extraer el proyecto e instalar las dependencias

1. Localizar el archivo .zip que contiene el código fuente de la aplicación y extraer su contenido. No ejecutar el proyecto directamente desde el archivo comprimido.
2. Identificar la carpeta mobile, que contiene el proyecto React Native desarrollado con TypeScript.
3. Abrir **PowerShell** dentro de la carpeta mobile.
   En nuestra configuración, la ruta utilizada fue:

   C:\Users\artem\OneDrive\Escritorio\AutoMind\mobile

   La ruta dependerá del lugar donde cada persona extraiga el proyecto.
4. Verificar que la carpeta contenga los archivos package.json y packag e-lock.json.
5. Ejecutar el siguiente comando:

   npm.cmd ci

6. Esperar hasta que termine la instalación de las dependencias antes de continuar.

El comando npm.cmdci instala las dependencias utilizando las versiones registradas en package-lock.json.

En PowerShell utilizamos npm.cmd porque la ejecución de npm intentaba abrir el archivo npm.ps1, bloqueado por la política de ejecución de scripts del equipo.

Abrir el proyecto Android en Android Studio

1. Abrir **Android Studio**.
2. En la pantalla inicial, seleccionar **Open**. Si ya existe otro proyecto abierto, utilizar **File >Open**.
3. Localizar la carpeta extraída de la aplicación y seleccionar:

   mobile\android

4. Abrir esa carpeta como proyecto Android.
5. Permitir que Android Studio reconozca el proyecto y comience la sincronización de Gradle. Si solicita configurar Java o instalar componentes del SDK, completar los apartados siguientes.

Configurar JDK 17 para Gradle

1. En Android Studio, dirigirse a:

   File >Settings >Build, Execution, Deployment >Build Tools >Gradle

2. Localizar el apartado **Gradle JDK**.
3. Seleccionar una instalación de **JDK 17**.
   En nuestro equipo utilizamos:

   C:\Program Files\Microsoft\jdk-17.0.20.101-hotspot

4. Si esa instalación no aparece en el listado, seleccionar **Add JDK** y buscar la carpeta donde se encuentra instalado JDK 17.
   Seleccionar la carpeta principal del JDK, sin entrar en bin.
5. Seleccionar **Apply** y luego **OK**.

La selección anterior configura el Java utilizado por las acciones de Gradle ejecutadas desde Android Studio.

Para ejecutar Gradle desde PowerShell, también configuramos JAVA_HOME en la terminal utilizada para compilar.

Instalar los componentes del SDK de Android

1. En Android Studio, abrir **Tools >SDK Manager**.
2. Comprobar la ubicación indicada en **Android SDK Location**.
   En nuestro equipo fue:

   C:\Users\artem\AppData\Local\Android\Sdk

3. En la pestaña **SDK Platforms**, comprobar que esté instalado:

   Android SDK Platform 36 -- Android 16

4. Abrir la pestaña **SDK Tools** y activar **Show Package Details** para visualizar las versiones disponibles.
5. Comprobar los componentes utilizados por aquella versión del proyecto:
   - **Android SDK Build-Tools:** 36.0.0.
   - **NDK (Side by side):** 27.1.12297006.
   - **CMake:** 3.22.1.
   - **Android SDK Platform-Tools**.
6. Si falta algún componente, seleccionarlo y pulsar **Apply**.
7. Aceptar las licencias correspondientes y esperar hasta que finalice la instalación.
8. Seleccionar **OK** para cerrar el administrador del SDK.

Estas versiones corresponden al proyecto que compilamos en esa conversación. Si se utiliza otra versión del código fuente, deben comprobarse los requisitos declarados en sus archivos de configuración Android.

Comprobar Java desde PowerShell

1. Abrir una terminal PowerShell dentro de mobile.
   Si la terminal todavía se encuentra dentro de mobile, ejecutar:

   cd android

2. Configurar JDK 17 para esa terminal:

   $env:JAVA_HOME = 'C:\Program Files\Microsoft\jdk-17.0.20.101-hotspot'

   Sustituir la ruta si JDK 17 está instalado en otra ubicación.

3. Consultar la configuración de Gradle:

   .\gradlew.bat --version

4. Comprobar que la información mostrada corresponda a Java 17.

La asignación de JAVA_HOME se mantiene durante esa sesión de PowerShell.

Corregir la selección de Java del daemon de Gradle

Durante nuestra configuración encontramos un archivo que todavía solicitaba Java 25. Por ello, seleccionar JDK 17 en Android Studio y configurar JAVA_HOME no era suficiente.

Este ajuste corresponde a ese problema concreto.

1. Comprobar si existe el archivo:

   mobile\android\gradle\gradle-daemon-jvm.properties

2. Si ese archivo todavía solicita Java 25, dejar su contenido de la siguiente manera, tal como hicimos en aquella configuración:

   toolchainVersion=17

3. Guardar el archivo.
4. Ejecutar nuevamente, desde mobile:

   .\gradlew.bat --version

5. Comprobar que aparezcan **Launcher JVM: 17** y una indicación de compatibilidad del daemon con Java 17.

Corrección del error de updateDaemonJvm También encontramos el siguiente error al ejecutar la tarea updateDaemonJvm:

Toolchain download repositories have not been configured.

Para resolverlo en aquella configuración, realizamos estos pasos:

1. Abrir el archivo Gradle ubicado directamente dentro de mobile:

   mobile\android\build.gradle

   Importante: En este caso se modifica el archivo del proyecto Android, no mobile.gradle.

2. Añadir al final del archivo, fuera de los bloques existentes:

   ```
   tasks.named("updateDaemonJvm") {
       toolchainPlatforms = []
   }
   ```

3. Guardar los cambios.

4. Desde mobile, ejecutar:

   .\gradlew.bat updateDaemonJvm --jvm-version=17

5. Esperar a que aparezca **BUILD SUCCESSFUL**.

Este procedimiento evita generar las URL de descarga de Java para otras plataformas y permite trabajar con el JDK instalado.

No es necesario repetir esta corrección si el proyecto ya tiene configurado Java 17 y no presenta ese error.

Compilar el APK de depuración

1. Confirmar que la terminal se encuentre dentro de:

   mobile\android

2. Ejecutar:

   .\gradlew.bat assembleDebug

3. Esperar hasta que termine la compilación.
   La primera ejecución puede descargar dependencias y compilar componentes nativos, por lo que puede tardar considerablemente más que las siguientes.

4. Verificar que aparezca:

   BUILD SUCCESSFUL

5. Localizar el APK generado en:

   mobile\android\app\build\outputs\apk\debug\app-debug.apk

Ejecutar la versión de desarrollo

El APK de depuración del proyecto original necesita Metro para cargar el código de la aplicación.

1. Conectar un teléfono Android con **Depuración USB** habilitada o iniciar un emulador.
2. Abrir una terminal dentro de mobile y ejecutar:

   npm.cmd start

3. Mantener esa terminal abierta.
4. En otra terminal, también dentro de mobile, ejecutar:

   npm.cmd run android -- --active-arch-only

5. Esperar la instalación y apertura de la aplicación en el dispositivo.

Este último comando permite compilar únicamente para la arquitectura del dispositivo utilizado durante las pruebas.

La versión ligera autónoma que preparamos después incluía el código dentro del APK y utilizaba una configuración de compilación diferente.

Referencias de las definiciones

Google Cloud. (s. f.). *¿Qué es la computación en la nube?*. https://cloud.google.com/learn/what-is-cloud-computing?hl=es-419

McDonald, P. (2008, 7 de abril). *Introducing Google App Engine + our new blog*. Google Cloud Platform Blog. https://cloudplatform.googleblog.com/2008/04/introducing-google-app-engine-our-new.html

Greene, D. (2016, 29 de septiembre). *Introducing Google Cloud*. Google Cloud Blog. https://cloud.google.com/blog/topics/inside-google-cloud/introducing-google-cloud

Ayodele, B. (2022, 10 de marzo). *How Spanner and BigQuery work together to handle transactional and analytical workloads*. Google Cloud Blog. https://cloud.google.com/blog/topics/developers-practitioners/how-spanner-and-bigquery-work-together-handle-transactional-and-analytical-workloads/

Dinesh, S. (2016, 27 de junio). *Creating a scalable API with microservices*. Google Cloud Blog. https://cloud.google.com/blog/products/gcp/creating-a-scalable-api-with-microservices/

Google Cloud. (2026, 5 de octubre). *Ubicaciones de Cloud*. https://cloud.google.com/about/locations?hl=es-419

Kava, J. (2016, 24 de marzo). *Google shares data center security and design best practices*. Google Cloud Blog. https://cloud.google.com/blog/products/gcp/google-shares-data-center-security-and-design-best-practices/

Vergadia, P. (2022, 8 de junio). *Infrastructure security in Google Cloud*. Google Cloud Blog. https://cloud.google.com/blog/topics/developers-practitioners/infrastructure-security-in-google-cloud

Canonical. (s. f.-a). *Security updates*. Ubuntu Security Documentation. https://documentation.ubuntu.com/security/security-updates/

Canonical. (s. f.-b). *Ubuntu Server documentation*. https://ubuntu.com/server/docs/

Canonical. (s. f.-c). *UEFI Secure Boot*. Ubuntu Security Documentation. https://documentation.ubuntu.com/security/security-features/platfo

rm-protections/secure-boot/

Google Cloud. (2024a). *Google infrastructure security design overview*. https://docs.cloud.google.com/docs/security/infrastructure/design

Google Cloud. (2024b). *How Google enforces boot integrity on production machines*. https://docs.cloud.google.com/docs/security/boot-integrity

Google Cloud. (2026). *How Google protects its production services*. https://docs.cloud.google.com/docs/security/production-services-protection

Kava, J. (2016, 24 de marzo). *Google shares data center security and design best practices*. Google Cloud Blog. https://cloud.google.com/blog/products/gcp/google-shares-data-center-security-and-design-best-practices/

Vergadia, P. (2022, 8 de junio). *Infrastructure security in Google Cloud*. Google Cloud Blog. https://cloud.google.com/blog/topics/developers-practitioners/infrastructure-security-in-google-cloud

Google Maps Platform. (s. f.). *Preguntas frecuentes sobre Google Maps Platform*. Google for Developers. https://developers.google.com/maps/faq?hl=es-419

Google Maps Platform. (s. f.). *APIs de Google Maps Platform por plataforma*. Google for Developers. https://developers.google.com/maps/apis-by-platform?hl=es-419

Firebase. (s. f.). *Firebase: Google's mobile and web app development platform*. Google. https://firebase.google.com/

Firebase. (s. f.-a). *Access data offline*. Google. https://firebase.google.com/docs/firestore/manage-data/enable-offline

Firebase. (s. f.-b). *Cloud Firestore*. Google. https://firebase.google.com/docs/firestore

Firebase. (s. f.-c). *Cloud Firestore locations*. Google. https://firebase.google.com/docs/firestore/locations

Firebase. (s. f.-d). *Firebase Authentication*. Google. https://firebase.g

oogle.com/docs/auth

Firebase. (s. f.-e). *Firebase Realtime Database*. Google. https://firebase.google.com/docs/database

Firebase. (s. f.-f). *Get started with Cloud Firestore Security Rules*. Google. https://firebase.google.com/docs/firestore/security/get-started

Firebase. (s. f.-g). *Understand Cloud Firestore billing*. Google. https://firebase.google.com/docs/firestore/pricing

Firebase. (s. f.-h). *Understand Firebase projects*. Google. https://firebase.google.com/docs/projects/learn-more

Microsoft. (s. f.-a). *Azure Cosmos DB overview*. Microsoft Learn. https://learn.microsoft.com/en-us/azure/cosmos-db/overview

Microsoft. (s. f.-b). *Change feed pull model in Azure Cosmos DB*. Microsoft Learn. https://learn.microsoft.com/en-us/azure/cosmos-db/nosql/change-feed-pull-model

Microsoft. (s. f.-c). *Introduction to Microsoft Entra External ID*. Microsoft Learn. https://learn.microsoft.com/en-us/azure/active-directory/external-identities/external-identities-overview

Microsoft. (s. f.-d). *What is Azure Database for PostgreSQL flexible server?*. Microsoft Learn. https://learn.microsoft.com/en-us/azure/postgresql/overview

Microsoft. (s. f.-e). *What is Azure SQL Database?*. Microsoft Learn. https://learn.microsoft.com/en-us/azure/azure-sql/database/sql-database-paas-overview

Firebase. (s. f.-a). *Cloud Firestore*. Google. https://firebase.google.com/docs/firestore

Firebase. (s. f.-b). *Firebase Authentication*. Google. https://firebase.google.com/docs/auth/

Google Cloud. (s. f.-a). *Artifact Registry overview*. https://docs.cloud.google.com/artifact-registry/docs/overview

Google Cloud. (s. f.-b). *Cloud Billing overview*. https://docs.cloud.google.com/billing/docs/concepts

Google Cloud. (s. f.-c). *Cloud Build documentation*. https://docs.cloud.google.com/build/docs

Google Cloud. (s. f.-d). *Cloud Run documentation*. https://docs.cloud.google.com/run/docs

Google Cloud. (s. f.-e). *Cloud Shell documentation*. https://docs.cloud.google.com/shell/docs

Google Cloud. (s. f.-f). *IAM overview*. https://docs.cloud.google.com/iam/docs/overview

Google for Developers. (s. f.). *Using OAuth 2.0 to access Google APIs*. Google. https://developers.google.com/identity/protocols/oauth2/

Firebase. (s. f.-a). *Administra sesiones de usuarios*. Google. https://firebase.google.com/docs/auth/admin/manage-sessions?hl=es-419

Firebase. (s. f.-b). *Verifica tokens de ID*. Google. https://firebase.google.com/docs/auth/admin/verify-id-tokens?hl=es-419

Google Cloud. (s. f.). *Contrato de entorno de ejecución del contenedor*. https://docs.cloud.google.com/run/docs/container-contract?hl=es-419

Google Cloud. (s. f.-a). *Container runtime contract*. https://docs.cloud.google.com/run/docs/container-contract

Google Cloud. (s. f.-b). *What is containerization?*. https://cloud.google.com/discover/what-is-containerization

Google Cloud. (s. f.). *¿Qué es la creación de contenedores? ¿Cómo funciona?*. https://cloud.google.com/discover/what-is-containerization?hl=es-419

Google Cloud. (s. f.). *Descripción general de Cloud Build*. https://docs.cloud.google.com/build/docs/overview?hl=es-419

Google Cloud. (s. f.). *Descripción general de Artifact Registry*. https://docs.cloud.google.com/artifact-registry/docs/overview?hl=es-419

Referencias

Araya Day, A. (s. f.). *Tutorial-App-II* [Repositorio de código]. GitHub. https://github.com/artemioadaysolvers/Tutorial-App-II

Google Cloud. (s. f.). *Cloud SQL overview*. Google Cloud Documentation. https://cloud.google.com/sql/docs/postgres/introduction

PostgreSQL Global Development Group. (s. f.-a). *About PostgreSQL*. https://www.postgresql.org/about/

PostgreSQL Global Development Group. (s. f.-b). *The SQL language*. PostgreSQL Documentation. https://www.postgresql.org/docs/current/sql.html

Android Developers. (s. f.). *Configure the base module*. Android Developers. https://developer.android.com/guide/app-bundle/configure-base

Google Cloud. (s. f.-a). *Authenticating users*. Google Cloud Documentation. https://docs.cloud.google.com/run/docs/authenticating/end-users

Google Cloud. (s. f.-b). *Manage revisions*. Google Cloud Documentation. https://docs.cloud.google.com/run/docs/managing/revisions

Google Cloud. (s. f.-c). *Secret Manager overview*. Google Cloud Documentation. https://docs.cloud.google.com/secret-manager/docs/overview

Google Cloud. (s. f.-d). *What is Cloud Run?* Google Cloud Documentation. https://docs.cloud.google.com/run/docs/overview/what-is-cloud-run